

Autogenesis: A Self-Evolving Agent Protocol

Wentao Zhang^{1*}, Zhe Zhao^{2*}, Haibin Wen^{3*}, Yingcheng Wu², Cankun Guo⁴,
Ming Yin^{2†}, Bo An^{1†}

¹ Nanyang Technological University ² Stanford University ³ City University of Hong Kong

⁴ University of Science and Technology of China

mingyin0312@gmail.com boan@ntu.edu.sg

Project Code: <https://github.com/DVampire/Autogenesis>

Abstract

Recent advances in LLM based agent systems have shown promise in tackling complex, long horizon tasks. However, existing agent protocols (e.g., A2A and MCP) under specify cross entity lifecycle and context management, version tracking, and evolution safe update interfaces, which encourages monolithic compositions and brittle glue code. We introduce **AUTOGENESIS PROTOCOL (AGP)**, a self evolution protocol that decouples what evolves from how evolution occurs. Its Resource Substrate Protocol Layer (RSPL) models prompts, agents, tools, environments, and memory as protocol registered resources¹ with explicit state, lifecycle, and versioned interfaces. Its Self Evolution Protocol Layer (SEPL) specifies a closed loop operator interface for proposing, assessing, and committing improvements with auditable lineage and rollback. Building on **AGP**, we present **AUTOGENESIS SYSTEM (AGS)**, a self-evolving multi-agent system that dynamically instantiates, retrieves, and refines protocol-registered resources during execution. We evaluate **AGS** on multiple challenging benchmarks that require long horizon planning and tool use across heterogeneous resources. The results demonstrate consistent improvements over strong baselines, supporting the effectiveness of agent resource management and closed loop self evolution.

1 Introduction

Recent advances in LLM-based agent systems have demonstrated significant potential in tackling complex, long-horizon tasks [38, 36, 3], yet static designs often prove insufficient against the diversity and stochasticity of real-world environments. Endowing agents with self-evolution capabilities has thus emerged as a critical avenue toward robust autonomy. However, existing implementations remain largely fragmented and ad hoc: components such as prompts, tools, and memory are tightly coupled to agent logic, shared standards are absent, and the lack of explicit lifecycle management and safe update interfaces introduces significant risks of runtime instability, preventing self-evolution from being composable, auditable, or systematically reproducible.

Although protocols such as MCP [2] and A2A [9] have standardized connectivity for model-tool invocation and inter-agent communication, they operate solely at the level of invocation and message passing, leaving internal resource states opaque. Neither provides mechanisms for lifecycle management, version lineage, or controlled state mutation, which are precisely the requirements of a closed-loop evolutionary system. Bridging this gap calls for a dedicated protocol addressing three essential properties: **Decoupling**, so that resources such as prompts, tools, and memory are managed as independent entities rather than tightly coupled code; **Safety & Auditability**, through strict version

*Equal contribution. First Author Contact: zhangwent963@gmail.com [†]Corresponding authors.

¹Unless otherwise specified, resources refer to instances of the five RSPL entity types: *prompt*, *agent*, *tool/MCP/skill*, *environment*, *memory* with agent *outputs/solutions*. *Tool* refers to local code-based tools, MCP tools, and skills.

control and rollback to ensure every evolutionary step is traceable and reversible; and **Formalism**, via standardized operators (e.g., reflect, propose, verify) that convert heuristic modifications into a rigorous control loop.

To address these challenges, we propose **AUTOGENESIS PROTOCOL (AGP)**, a two-layer protocol architecture that formally decouples the evolutionary substrate from the evolutionary logic. The central design principle is to standardize resource representations, enabling uniform application of optimization algorithms [40, 31, 14] across heterogeneous agent components. The **Resource Substrate Protocol Layer (RSPL)** constitutes the substrate of evolution, modeling prompts, agents, tools, environments, and memory systems as protocol-registered resources endowed with explicit state, lifecycle, and versioned interfaces, thereby rendering them well-defined objects amenable to systematic observation and controlled manipulation. The **Self-Evolution Protocol Layer (SEPL)** establishes a closed-loop operator interface grounded in control theory, specifying a set of atomic operators that formally govern the evolution cycle and guarantee that every self-modification is fully auditable and subject to strict safety constraints. Building upon this protocol, we instantiate **AUTOGENESIS SYSTEM (AGS)**, a self-evolving multi-agent system system that dynamically registers, retrieves, and refines protocol resources at runtime. Empirical evaluation on a suite of challenging benchmarks, including GPQA [28], AIME, GAIA [19], HLE [24], and LeetCode [16], demonstrates that **AGS** achieves consistent and substantial improvements over strong baselines, validating the efficacy of principled resource management and closed-loop self-evolution. The contributions of this work are threefold:

- We propose **AUTOGENESIS PROTOCOL (AGP)**, a two-layer self-evolution protocol decoupling evolutionary substrate from logic. RSPL endows resources with explicit state, lifecycle, and versioned interfaces; SEPL governs the evolution cycle via a closed-loop operator interface with auditable lineage and rollback.
- We present **AUTOGENESIS SYSTEM (AGS)**, a self-evolving multi-agent system system that dynamically registers, retrieves, and refines protocol resources at runtime, demonstrating the practical viability of protocol-driven self-evolution.
- We conduct empirical evaluation on five challenging benchmarks (GPQA, AIME, GAIA, HLE, and LeetCode), demonstrating consistent and substantial improvements over strong baselines and validating the efficacy of principled resource management and closed-loop evolution.

2 Related Work

2.1 LLM-based Agent Systems and Protocols

LLM-based agent systems have demonstrated strong capabilities in complex, long-horizon tasks requiring multi-step reasoning and external tool interaction [28, 19, 38, 36, 29], with LLMs serving as centralized decision-making modules that decompose tasks and invoke tools to act on the environment. However, most existing frameworks treat prompts, tools, and memory as tightly coupled internal components: tools are manually curated fixed modules integrated directly into the agent pipeline [26, 29, 4], limiting systematic reuse and controlled adaptation as task requirements evolve. Efforts such as Anthropic’s MCP [1] and Google’s A2A protocol have standardized model-tool interaction and inter-agent communication at the level of invocation and message passing, but leave the internal state of agents and resources opaque, providing no mechanisms for managing resource lifecycles, tracking version lineage, or constraining state mutations over time. In contrast, our approach treats prompts, agents, local code-based tools, MCP tools, and skills [2] as protocol-registered entities endowed with explicit interfaces and versioned state, thereby supporting dynamic instantiation, controlled refinement, and auditable evolution throughout execution.

2.2 Self-Evolution and Optimization of Agent Components

A parallel line of work investigates iterative agent improvement via gradient-free methods such as TextGrad [40], which treat natural language feedback as a gradient signal [25, 44], and reinforcement learning approaches such as Reinforce++ [13] and GRPO [31], which frame agent components as policies optimized via evaluation rewards [33, 18, 41]. More recent frameworks such as EvoAgentX [35] and Hermes Agent [20] further pursue self-evolving agent workflows, autonomously constructing and refining multi-agent pipelines or skill libraries from interaction history. Despite this progress, these approaches focus on optimizing a narrow subset of agent components, typically prompts or task workflows, and do not provide a unified abstraction for managing the full spectrum of agent-internal entities including prompts, tools, and environments. Updates are applied directly

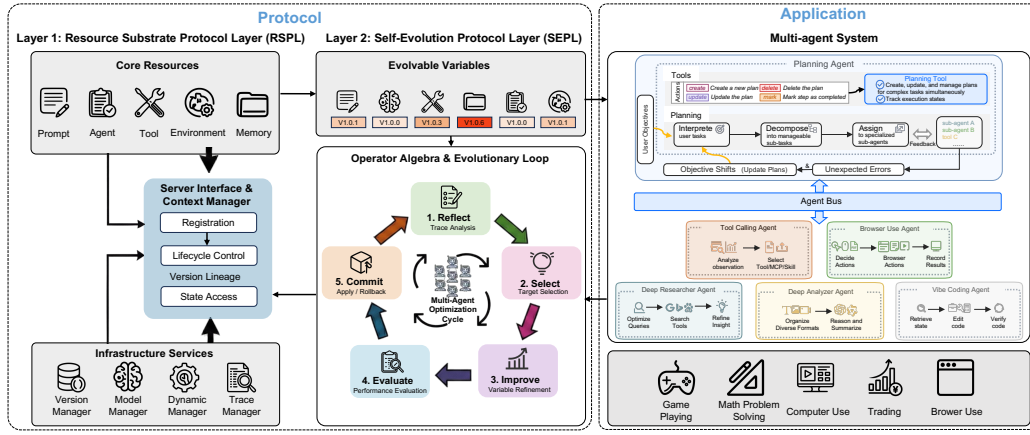


Figure 1: The Autogenesis protocol and system architecture.

without lifecycle control, version tracking, or rollback, precluding safe and auditable evolution. Our approach addresses this limitation via a two-layer architecture that exposes all agent components as protocol-registered resources governed by a principled closed-loop operator interface.

3 Autogenesis Protocol

This section describes the **AGP** specification, covering the resource substrate and the evolution operator interface. The concrete system instantiation built upon this protocol is presented in the next section. Despite growing interest in self-evolving agents [7], most systems remain engineered in an ad hoc manner and lack a shared protocol standard that makes evolution composable, auditable, and interoperable. As shown in Figure 1., we introduce **AGP**, a two-layer self-evolution protocol. The *Resource Substrate Protocol Layer (RSPL)* specifies the evolvable substrate, namely which resources may change and how they are represented, versioned, and accessed. The *Self-Evolution Protocol Layer (SEPL)* specifies the evolution logic, namely how updates are proposed, assessed, and committed through a safe operator interface. Inspired by interface standardization efforts in agent tooling, this separation cleanly decouples *what evolves* from *how evolution occurs*, enabling modularity, traceability, and safety-preserving evolution across components.

3.1 Layer 1: Resource Substrate Protocol Layer

The Resource Substrate Protocol Layer (RSPL) defines the evolvable substrate as a set of protocol-registered resources with explicit state, lifecycle, and version lineage. We identify five entity types as a minimal yet expressive common denominator across modern agent stacks, providing a uniform target space on which SEPL can operate: (i) *instructions* (Prompt), (ii) *decision policies* (Agent), (iii) *actuation interfaces* (Tool), encompassing local code-based tools, MCP tools [1], and agent skills [2], (iv) *task/world dynamics* (Environment), and (v) *persistent state* (Memory). Crucially, resources in RSPL are *passive*, meaning they encapsulate no optimization logic, cannot self-modify, and change state only through controlled operations mediated by interfaces and invoked by higher layers. This separation decouples agent logic from task-specific instructions and capability bundles [37, 11, 5], enabling the same policy to be deployed across tasks with different resource configurations.

3.1.1 Infrastructure Services

A self-evolution protocol requires reliable foundational support in which model access remains consistent as components are swapped, every state transition is traceable and reversible, resources persist across sessions and can be safely hot-swapped, and execution behavior is observable for diagnosis and improvement. To meet these requirements, RSPL provides four cross-cutting infrastructure services: (i) A **model manager** standardizes LLM API calls across heterogeneous providers, including Anthropic, OpenAI, Google, xAI, and OpenRouter, and supports routing and fallback to ensure consistent model access as resources evolve. (ii) A **version manager** maintains immutable snapshots and version lineage, enabling rollback, branching, and auditability at every state transition. (iii) A **dynamic manager** handles serialization and hot-swapping of resource configurations at runtime without restarting the agent system. (iv) A **trace manager** captures fine-grained execution traces for interpretability, debugging, and retrospective optimization.

3.1.2 Core Entities

Definition 3.1 (Resource Entity). A resource entity and its type-level collection are defined as:

$$\begin{aligned} e_{\tau,i} &= (n_{\tau,i}, d_{\tau,i}, \phi_{\tau,i}, g_{\tau,i}, m_{\tau,i}), \\ \mathcal{E}_{\tau} &= \{e_{\tau,i} \mid i \in \mathcal{I}_{\tau}\}, \end{aligned} \quad (1)$$

where $\mathcal{T} = \{\text{PROMPT}, \text{AGENT}, \text{TOOL}, \text{ENV}, \text{MEM}\}$ is the set of RSPL entity types, $\tau \in \mathcal{T}$ indexes the type, $\mathcal{I}_{\tau}$ is the index set for instances of type τ , and $i \in \mathcal{I}_{\tau}$ indexes an individual instance. $n_{\tau,i}$ is a unique name, $d_{\tau,i}$ a short description, $\phi_{\tau,i} : \mathcal{X}_{\tau} \rightarrow \mathcal{Y}_{\tau}$ an input-to-output mapping, $g_{\tau,i} \in \{0, 1\}$ an evolvability marker, and $m_{\tau,i}$ an auxiliary metadata dictionary.

To support resource registration, unified management, and instantiation, RSPL stores a serializable registration record for each resource instance.

Definition 3.2 (Resource Registration Record). A resource registration record and its type-level collection can be represented as:

$$\begin{aligned} c_{\tau,i} &= (e_{\tau,i}, v_{\tau,i}, \eta_{\tau,i}, \theta_{\tau,i}, \mathcal{F}_{\tau,i}), \\ \mathcal{C}_{\tau} &= \{c_{\tau,i} \mid i \in \mathcal{I}_{\tau}\}, \end{aligned} \quad (2)$$

where $\tau \in \mathcal{T}$ indexes the entity type and $i \in \mathcal{I}_{\tau}$ indexes an individual instance. Here $e_{\tau,i}$ is the resource entity tuple defined in Theorem 3.1, $v_{\tau,i} \in \mathbb{V}$ is a version string, $\eta_{\tau,i}$ is an implementation descriptor (e.g., import path, class definition, or source-code string), $\theta_{\tau,i}$ are instantiation parameters (e.g., constructor arguments), and $\mathcal{F}_{\tau,i}$ is a set of exported representations used by LLMs to interact with the resource (e.g., function-calling schema, plain text, and structured argument schema).

Definition 3.3 (Protocol-registered resource). For each entity type τ , let $\mathcal{R}_{\tau}$ denote the type-specific registry of protocol-registered resources, and let $\mathcal{R} = \bigcup_{\tau} \mathcal{R}_{\tau}$ denote the corresponding global registry. RSPL associates each entity type τ with a dedicated context manager $\mathcal{M}_{\tau}$ and a server-exposed interface $\mathcal{A}_{\tau}$. We represent the type-level registered resource as

$$r_{\tau} = (\mathcal{C}_{\tau}, \mathcal{M}_{\tau}, \mathcal{A}_{\tau}), \quad (3)$$

where each $c_{\tau,i} \in \mathcal{C}_{\tau}$ denotes a registration record as defined in Theorem 3.2. The context manager $\mathcal{M}_{\tau}$ maintains the record collection $\mathcal{C}_{\tau}$ and the version lineage associated with type τ , while implementing lifecycle and update operations over these records. The server-exposed interface $\mathcal{A}_{\tau}$ encapsulates $\mathcal{M}_{\tau}$ and provides a unified external interface by delegating incoming requests to the corresponding context-manager routines.

Context manager and server interface. Each resource type is governed by a context manager, which serves as the management plane. It maintains a registry of materialized resources, preserves versioned histories for restoration, and supports *contract generation* by producing a consolidated capability specification. This specification reduces prompt bloat and enables *context engineering* through controlled injection. For tools, the contract takes a `skills.md`-style form [2] that enumerates actions, arguments, and usage constraints. The context-manager API provides operators for lifecycle management (`init`, `build`, `retrieval` (`list`, `get`), versioning (`update`, `restore`), execution (`run`), and serialization (`save_to_json`, `load_from_json`, `save_contract`, `load_contract`). The server interface encapsulates this internal complexity behind a uniform set of endpoints with consistent request and response semantics, providing a single control plane for safe and version-aware interactions with RSPL resources. Full specifications are in Section E.2.2.

3.2 Layer 2: Self-Evolution Protocol Layer (SEPL)

The Self-Evolution Protocol Layer (SEPL) formalizes agentic system evolution as a generalized optimization problem over a heterogeneous state space, modeling evolutionary dynamics as a state transition function governed by a strictly typed operator algebra. By mediating all state mutations through standardized RSPL interfaces, SEPL guarantees that evolution is traceable, reversible, and safe-by-construction. While this paper focuses on the reflection-driven optimizer as the primary instantiation, the same state manipulation primitives also accommodate textual-gradient methods such as TextGrad [40] and reinforcement learning approaches such as GRPO [31] and Reinforce++ [14].

3.2.1 Evolvable Variables

To transition from heuristic adaptation to a systematic evolution protocol, we introduce the concept of *variable lifting*. This abstraction projects discrete, heterogeneous RSPL resources (e.g., tool code, system prompts) onto a unified representation of evolvable variables. This formalism offers significant theoretical advantages by homogenizing the interaction surface for evolutionary operators and rigorously delineating the trainable subspace via an explicit learnability mask.

Definition 3.4 (Evolvable Variable Set). We define the universal set of evolvable variables as $\mathcal{V}_{\text{evo}} = (\bigcup_{\tau \in \mathcal{T}} \mathcal{E}_{\tau}) \cup \{y\}$, where $\mathcal{E}_{\tau}$ denotes the set of resource entities of type τ governed by the RSPL. The element y encapsulates execution artifacts, specifically final outputs and reasoning traces, which constitute the observational basis for retrospective optimization. Furthermore, each variable $v \in \mathcal{V}_{\text{evo}}$ is associated with a binary learnability constraint $g_v \in \{0, 1\}$, thereby strictly defining the trainable parameter subspace $\Theta = \{v \in \mathcal{V}_{\text{evo}} \mid g_v = 1\}$.

3.2.2 Operator Algebra

To systematically govern state transitions over $\mathcal{V}_{\text{evo}}$, we introduce the notion of a *SEPL operator*: a typed, composable function that reads the current evolvable state together with auxiliary signals, produces an updated state, and emits signals for downstream operators. Formalizing evolution as an algebra of such operators ensures that every modification is interface-mediated, auditable, and reversible, regardless of the specific optimization strategy instantiated.

Definition 3.5 (SEPL Operator). Let $\mathcal{V}_{\text{evo}}$ be the evolvable variable set and $\mathcal{P}$ a *message space* carrying auxiliary signals (e.g., traces, hypotheses, gradients, or reward signals) passed between operators. A *SEPL operator* is a function

$$f : \mathcal{V}_{\text{evo}} \times \mathcal{P}_{\text{in}} \rightarrow \mathcal{V}'_{\text{evo}} \times \mathcal{P}_{\text{out}}, \quad (4)$$

where $\mathcal{P}_{\text{in}}, \mathcal{P}_{\text{out}} \subseteq \mathcal{P}$ are the incoming and outgoing message types, and $\mathcal{V}'_{\text{evo}}$ is the updated evolvable state. Operators are *composable*: the output $(\mathcal{V}'_{\text{evo}}, \mathcal{P}_{\text{out}})$ of one operator serves as the input to the next, enabling the construction of an evolutionary pipeline $f_n \circ \dots \circ f_1$.

3.2.3 Evolutionary Loop

Given an initial evolvable state $\mathcal{V}_{\text{evo}}^{(0)}$ and an empty message $\mathcal{P}^{(0)} = \emptyset$, the evolutionary loop at each iteration t applies a sequence of operators $f_1, \dots, f_n$ in composition:

$$(\mathcal{V}_{\text{evo}}^{(t+1)}, \mathcal{P}^{(t+1)}) = (f_n \circ \dots \circ f_1)(\mathcal{V}_{\text{evo}}^{(t)}, \mathcal{P}^{(t)}), \quad (5)$$

where each f_i reads the current state and incoming messages, produces an updated state and outgoing messages consumed by f_{i+1} . The loop repeats until convergence or budget exhaustion. By routing all state mutations through RSPL interfaces, each transition is versioned and reversible, guaranteeing that evolution is *grounded* in execution data, *traceable* through versioned updates, and *safe-by-construction*. For example, the reflection optimizer instantiates this loop with five operators: REFLECT maps execution traces and current state to causal failure hypotheses, SELECT identifies target evolvable entities from the current state and hypotheses, generating concrete modification proposals, IMPROVE applies proposals via RSPL interfaces to yield a candidate state, EVALUATE scores the candidate against the objective and safety invariants, and COMMIT conditionally accepts or rolls back the transition. Full pseudocode for all instantiations is in Section E.3.

4 Autogenesis System

4.1 Autogenesis System Architecture

As shown in Figure 1, building on **AGP**, we instantiate the two-layer protocol into **AGS**, a self-evolving multi-agent system. A self-evolving system requires that agents, tools, and coordination structures remain dynamically modifiable at runtime, which is fundamentally incompatible with monolithic controllers or hard-wired pipelines that tightly couple execution logic to agent identity. To satisfy this requirement, we adopt a *bus interaction model* [37, 11]: the planning agent and all sub-agents register as first-class participants on a shared *Agent Bus*, and all inter-agent communication is mediated exclusively through standardized bus messages. This decoupling enables loose coupling, transparent observability, and concurrent sub-agent execution, while allowing any participant to be replaced or evolved without disrupting the rest of the system. Throughout all configurations, prompts, tools, and agents are treated as *first-class RSPL resources* with explicit lifecycle and version lineage. The system operates through three interleaved mechanisms:

Orchestration via Plan Generation. Upon receiving a task via the bus, the planning agent is responsible solely for planning and coordination and does not execute subtasks directly. It produces a structured `plan.md` artifact comprising five components: the original task description, a to-do list of subtask steps each assigned to a designated sub-agent (e.g., deep researcher agent, browser-use agent, deep analyzer agent and vibe coding agent), an execution flowchart, a running execution history, and a final result summary. The planning agent dispatches subtasks to the designated sub-agents via the

bus, executing independent subtasks concurrently and dependent ones sequentially, and collects all results through the bus before proceeding to the next round.

Concurrent Sub-Agent Execution and Iterative Re-planning. Upon receiving a dispatched subtask, each sub-agent independently retrieves relevant prompt and tool resources from the RSPL registry, executes tool calls, and writes results and reasoning traces to shared memory. Multiple sub-agents execute concurrently, as the bus decouples dispatch from completion. Once a round concludes, the planning agent collects outputs via the bus, updates `plan.md`, and determines whether the task is complete or a further round of decomposition is required. This collect-and-replan loop continues until the termination condition is met. As a complementary pattern, **AGS** also supports *agent-as-tool* composition, in which a sub-agent is wrapped behind a standard RSPL tool schema and invoked directly by a tool-calling agent, enabling lightweight collaboration without bus-level orchestration.

Self-Evolution. Interleaved with the bus coordination loop, **AGS** invokes the SEPL evolutionary loop whenever execution traces signal correctable failures or suboptimal performance. The loop applies a sequence of SEPL operators to reflect, select, improve, evaluate, and commit resource modifications as versioned RSPL transitions with auditable lineage and rollback. As an example instantiation, the reflection optimizer (Algorithm 1) reflects on execution traces to derive causal failure hypotheses, generates modification proposals (e.g., prompt text, tool source code, MCP configurations, or skill definitions), and commits accepted updates only after evaluating candidates against the task objective. Successful updates are immediately available to all sub-agents in subsequent bus rounds, ensuring that evolution remains traceable throughout the agent lifetime.

Beyond the reflection optimizer, our implementation supports additional optimization strategies that map naturally onto the same SEPL operator interface. *TextGrad* [40] instantiates the proposal and improvement operators as a gradient-informed text editor, treating natural-language feedback as a textual gradient applied to string variables. *Reinforce++* / *GRPO* [14, 31, 23, 45, 30] adopt a reinforcement-learning perspective, treating evolvable variables as policies optimized via policy-gradient estimates against evaluation rewards. These strategies demonstrate that SEPL is sufficiently general to accommodate inference-time reflection optimization, textual-gradient-based string updates, and reward-driven policy optimization within a unified protocol.

5 Empirical Studies

In this section, we present empirical results of deploying **AGS** across various challenging benchmarks with **AGP** to demonstrate its comprehensive capabilities.

Benchmark Instruction. We organize our evaluation into three categories. (i) **Scientific and Mathematical Benchmarks.** **GPQA-Diamond** (198 questions) presents graduate-level STEM multiple-choice questions (biology, chemistry, and physics) under a closed-book, non-retrieval protocol, measuring deep scientific understanding and multi-step reasoning. **AIME24** and **AIME25** each consist of 30 competition-level mathematics problems requiring exact integer answers, measuring long-horizon symbolic reasoning and arithmetic precision. (ii) **General Agent Benchmarks.** **GAIA** [19] includes a Validation split (165 tasks) and a Test split (300 tasks), each specifying a real-world, multi-step objective requiring planning and tool use (e.g., web browsing, document operations), measured by task-completion accuracy across three difficulty tiers. **Humanity’s Last Exam (HLE)** [24] comprises extremely difficult expert-level questions spanning mathematics, science, and humanities, measuring the agent’s capacity for deep reasoning at the boundary of human expert knowledge. (iii) **Self-Evolving Code Agent Benchmark.** Existing code benchmarks evaluate one-shot correctness under fixed model capability and therefore cannot measure an agent’s self-evolution capability during inference. To directly assess this self-evolution capability, we construct an in-house **LeetCode** benchmark of 100 recently released problems across diverse algorithmic categories (e.g., arrays, trees, linked lists), with reduced data contamination. The agent solves each problem in multiple languages (Python, C++, Java, Go, Kotlin), and we report acceptance rate, test-case pass rate, runtime efficiency, and human-relative performance metrics.

5.1 Experiments on Scientific and Mathematical Benchmarks

Experiment Setting. We evaluate **AGS** on GPQA-Diamond, AIME24, and AIME25, focusing on evolving prompts and agent outputs (problem solutions). Since these benchmarks primarily test reasoning capability rather than tool use, we exclude external tools and conduct a controlled comparison across three evolution strategies: *Prompt-Evo*, *Solution-Evo*, and *Prompt-Solution-Joint-*

Table 1: Scientific and Mathematical Benchmarks. Table 2: GAIA Validation and Test Benchmarks.

Agent	GPQA	AIME24	AIME25
gpt-4o			
Vanilla	47.98	13.34	6.67
Prompt-Evo	53.81	13.34	13.34
Solution-Evo	53.53	16.67	13.34
PS-Joint-Evo	58.08	16.67	13.34
Improvement(%)	21.05↑	24.97↑	100↑
gpt-4.1			
Vanilla	65.15	23.34	20.00
Prompt-Evo	68.68	33.33	23.33
Solution-Evo	68.68	36.67	30.00
PS-Joint-Evo	67.67	40.00	33.33
Improvement(%)	3.87↑	71.38↑	66.65↑
grok-4.1-fast			
Vanilla	83.33	96.67	90.00
Prompt-Evo	83.84	96.67	93.33
Solution-Evo	87.81	96.67	90.00
PS-Joint-Evo	89.34	96.67	96.67
Improvement(%)	7.21↑	0.00	7.41↑
claude-sonnet-4.5			
Vanilla	78.28	76.67	73.33
Prompt-Evo	79.79	86.67	90.00
Solution-Evo	80.30	80.00	90.00
PS-Joint-Evo	81.44	86.67	90.00
Improvement(%)	4.04↑	13.04↑	22.73↑
gemini-3-flash-preview			
Vanilla	88.38	83.33	83.33
Prompt-Evo	88.89	93.33	86.67
Solution-Evo	87.88	93.33	90.00
PS-Joint-Evo	90.40	93.33	93.33
Improvement(%)	2.28↑	12.00↑	12.00↑

Agent	Level1	Level2	Level3	Avg.
Validation				
HF ODR [15]	67.92	53.49	34.62	55.15
o3-DR [21]	74.29	69.06	47.60	67.36
DeSearch [6]	90.57	72.01	38.46	72.73
Co-Sight [42]	86.79	73.26	42.31	72.73
Manus [32]	86.50	70.10	57.69	73.90
AWorld [39]	88.68	77.91	53.85	77.58
Langfun [8]	88.68	80.23	57.69	79.39
Skywork [43]	92.45	83.72	57.69	82.42
agent-2030	96.23	90.70	57.69	87.27
Alita [27]	88.68	89.53	76.92	87.27
Vanilla	92.45	88.37	88.46	89.70
Agent-Evo	96.23	93.02	88.46	93.33
Improvement(%)	4.09↑	5.26↑	0.00	4.05↑
Test				
o4-mini-DR [21]	67.59	59.10	44.28	59.30
JoyAgent [17]	77.42	67.30	46.94	67.11
o3-DR [21]	79.42	68.97	47.48	68.70
Langfun [8]	84.95	73.58	48.98	73.09
Alita [27]	92.47	71.70	55.10	75.42
DeSearch [6]	91.40	75.47	61.22	78.07
h2o [10]	89.25	79.87	61.22	79.73
Su-Zero-Ultra	93.55	77.36	65.31	80.40
AWorld [39]	95.70	81.13	57.14	81.73
HALO [12]	94.62	84.91	69.39	85.38
ToolOrchestra [34]	95.70	82.39	87.76	87.38
openJiuwen [22]	98.92	88.68	87.76	91.69
Vanilla	91.40	77.36	61.22	79.07
Agent-Evo	98.92	85.53	81.63	89.04
Improvement(%)	8.23↑	10.56↑	33.34↑	12.61↑

Figure 2: Performance comparison on the HLE full set benchmark [46]. *Evo*. We evaluate across diverse mainstream language models using the reflection optimizer with up to 3 rounds, after which the final agent output is taken as the solution. Performance is measured by exact-match accuracy, requiring the selected option to match the ground-truth answer for GPQA-Diamond, and numerical output to exactly match the reference integer for AIME24 and AIME25.

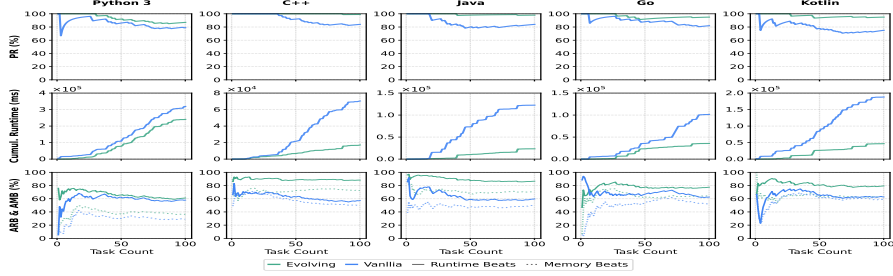
Results and Analysis. Table 1 reveals four key observations. (i) **Self-evolution yields consistent gains, with greater benefit for weaker models.** Weaker models gain substantially: gpt-4.1 improves by 71.4% on AIME24 and 66.7% on AIME25 under PS-Joint-Evo, and claude-sonnet-4.5 gains 13.0% and 22.7% respectively. Stronger models also benefit, albeit more modestly: gemini-3-flash-preview (vanilla 88.4% GPQA-Diamond, 83.3% AIME24/25) improves by 2.3%, 12.0%, and 12.0%, consistent with diminishing headroom at higher baselines. (ii) **PS-Joint-Evo consistently outperforms single-strategy evolution.** For gpt-4.1 on AIME24, Prompt-Evo reaches 33.3% and Solution-Evo 36.7%, while PS-Joint-Evo reaches 40.0%, confirming that prompt and solution refinement address complementary failure modes. (iii) **Math benchmarks benefit more than science QA.** AIME24/25 show larger relative gains than GPQA-Diamond across all models: for gpt-4.1, AIME24 improves by 71.4% versus 3.9% on GPQA-Diamond. Long-horizon symbolic reasoning exposes more intermediate failure points amenable to reflection, whereas closed-book science QA relies more on factual recall. (iv) **Ceiling effects limit gains on saturated benchmarks.** grok-4.1-fast reaches 96.7% on AIME24 under vanilla, leaving negligible headroom and yielding no gain from evolution. On GPQA-Diamond and AIME25 where its baselines are lower (83.3% and 90.0%), it still improves by 7.2% and 7.4%, confirming that self-evolution is most effective when sufficient headroom exists. Overall, PS-Joint-Evo is the preferred strategy when inference budget permits, as it addresses complementary failure modes simultaneously. For cost-constrained deployment, evolution budgets are best allocated to weaker models, harder tasks, or low-confidence samples. In near-saturated settings, adaptive triggering based on confidence or task difficulty is more effective than fixed-budget evolution.

5.2 Experiments on General Agent Benchmarks

Experiment Setting. For both GAIA and HLE, we focus on evolving agents (including agent prompts, tool implementations, and agent code), as these benchmarks primarily demand tool-augmented multi-

Table 3: Model performance on the Self-Evolving Code Agent Benchmark with **AGS** self-evolution.

Model	Capability metrics							Efficiency metrics			Human metrics		
	PR	TLE	MLE	CE	RE	WA	TO	RpE	AR (ms)	AM (MB)	APC	ARB (%)	AMB (%)
Python3													
deepseek-v3.2	34	1	1	0	23	8	1	36	1806.79	55.91	640.38	63.04	25.81
grok-4.1-fast	73	9	0	0	3	13	0	3	1860.90	56.02	741.60	49.92	30.15
claude-4.5-sonnet	42	37	2	0	0	10	8	1	880.98	45.16	702.64	61.06	22.12
claude-4.5-opus	82	9	0	0	0	5	3	1	1559.87	70.77	749.45	64.77	32.70
gemini-3-flash-preview	79	4	0	0	2	14	1	0	1376.19	56.59	750.89	73.28	36.62
+ Solution-Evo	87	3	0	0	1	9	0	0	1269.39	59.08	750.98	70.29	42.15
Improvement(%)	10.1↑	25.0↑	0	0	50↑	35.7↑	100↑	0	7.8↑	4.4↓	0.0	4.1↓	15.1↑
C++													
deepseek-v3.2	11	1	0	30	0	6	4	43	158.73	163.59	605.82	73.11	74.05
grok-4.1-fast	79	9	0	1	0	5	2	4	428.32	223.68	748.61	58.57	46.67
claude-4.5-sonnet	41	42	2	0	1	9	2	3	379.68	179.86	710.59	56.17	50.84
claude-4.5-opus	85	6	0	0	0	6	1	2	382.45	184.22	758.21	64.06	55.58
gemini-3-flash-preview	84	2	0	2	1	10	0	1	266.04	168.93	743.31	68.02	59.24
+ Solution-Evo	99	0	0	0	0	1	0	0	142.60	148.43	749.86	88.99	73.14
Improvement(%)	17.9↑	100↑	0	100↑	100↑	90↑	0	100↑	46.4↑	12.1↑	0.9↓	30.8↑	23.5↑
Java													
deepseek-v3.2	11	0	0	47	1	1	5	35	72.91	143.63	481.45	57.37	32.71
grok-4.1-fast	73	5	0	5	0	12	1	4	227.45	136.80	746.23	52.98	41.97
claude-4.5-sonnet	41	40	1	1	0	15	0	1	161.49	130.54	679.41	58.04	46.22
claude-4.5-opus	87	4	1	0	0	6	1	1	188.63	134.27	748.63	59.54	55.61
gemini-3-flash-preview	84	0	0	2	2	9	1	2	125.04	126.09	752.86	71.03	59.18
+ Solution-Evo	98	1	0	0	0	1	0	0	96.30	120.00	751.09	88.33	72.38
Improvement(%)	16.7↑	0	0	100↑	100↑	88.9↑	100↑	100↑	23.0↑	4.8↑	0.2↑	24.4↑	22.3↑
Go													
deepseek-v3.2	7	0	0	39	0	0	1	53	112.71	12.59	709.57	62.73	54.36
grok-4.1-fast	69	3	0	16	0	4	3	5	194.90	23.26	755.43	66.83	62.44
claude-4.5-sonnet	44	41	0	0	0	13	0	2	222.64	19.71	712.55	57.09	53.32
claude-4.5-opus	84	5	0	0	0	9	0	2	162.50	19.95	744.45	72.91	63.00
gemini-3-flash-preview	82	1	0	9	0	7	0	1	139.22	22.01	739.46	76.22	63.48
+ Solution-Evo	95	0	0	0	0	5	0	0	111.64	18.35	754.17	81.52	67.94
Improvement(%)	15.9↑	100↑	0	100↑	0	28.6↑	0	100↑	19.8↑	16.6↑	2.0↓	7.0↑	7.0↑
Kotlin													
deepseek-v3.2	7	2	0	0	0	0	1	48	59.29	62.27	793.57	58.33	74.56
grok-4.1-fast	62	2	0	22	0	8	2	4	307.45	75.45	759.55	78.12	72.83
claude-4.5-sonnet	42	36	1	8	1	10	1	1	192.62	78.49	757.64	81.59	77.79
claude-4.5-opus	83	4	0	5	0	5	0	3	210.47	76.60	750.98	83.18	76.53
gemini-3-flash-preview	75	2	0	8	1	10	2	2	171.99	72.80	760.43	83.49	79.07
+ Solution-Evo	95	1	0	0	0	4	0	0	122.83	77.88	749.38	83.58	67.21
Improvement(%)	26.7↑	50↑	0	100↑	100↑	60↑	100↑	100↑	28.6↑	7.0↓	1.5↓	0.1↑	15.0↓

Figure 3: Performance comparison of evolving and vanilla **AGS** within-inference.

step reasoning rather than pure deductive inference. Our system deploys a top-level planning agent ($m = 30$) coordinating four specialized sub-agents (deep researcher, browser-use, deep analyzer, tool calling agent and vibe coding agent, each with $m = 20$ using gemini-3.1-pro-preview), where m denotes the maximum reasoning steps. Agent self-evolution is driven by the vibe coding agent, which iteratively refines agent prompts and code through the SEPL reflection optimizer, with evolved agents registered as versioned RSPL resources and reused across subsequent tasks. For GAIA, we report Pass@1 accuracy at each difficulty tier (Level 1–3) and the overall average on both validation and test splits. For HLE, we follow the official evaluation protocol using o3-mini as the judge.

Results and Analysis. Table 2 and Figure 2 reveal three key observations. (i) **AGS achieves highly competitive performance among systems with comparable backbone models.** On GAIA, AGS attains strong results on both Test and Validation, reaching 89.04% on Test and the best reported Validation score of 93.33% among all listed systems. Although openJiuwen achieves a higher GAIA Test score, it relies on substantially stronger backbone models, which are orthogonal to the evolution protocol itself. On HLE, AGS ranks second overall, outperforming all systems except Claude Mythos Preview, which similarly benefits from a more capable frontier backbone. These comparisons suggest that the remaining gaps are primarily associated with backbone strength rather than the proposed self-evolution protocol. (ii) **Agent evolution yields the largest gains on hard tasks.** On GAIA Test, Agent-Evo improves the vanilla baseline by 12.6% on average, with gains increasing

with task difficulty. Improvements range from 8.2% on Level 1 to 33.3% on Level 3, indicating that harder tasks expose more correctable failure modes for iterative refinement, while easier tasks leave less room for improvement. This trend mirrors the headroom pattern observed in the math benchmarks. (iii) **Self-evolution generalizes to open-ended agent tasks.** GAIA requires coherent state management across multi-domain transitions, such as from browser retrieval to file analysis, while HLE demands expert-level multi-step reasoning. By registering prompts, agent code, and tools as versioned RSPL resources, **AGS** preserves task-critical state and reuses evolved capabilities across subsequent subtasks. Overall, these results show that **AGP**’s self-evolution protocol improves difficult agent tasks, remains competitive among systems with comparable backbone models, and extends from closed-form reasoning to complex, tool-intensive agent scenarios.

5.3 Experiments on Self-Evolving Code Agent Benchmark

Experiment Setting. Existing code generation benchmarks evaluate one-shot generation and do not measure an agent’s ability to iteratively improve solutions during inference. To address this, we construct a benchmark based on the LeetCode online judge using 100 recently released problems to mitigate contamination (details in Section D). We compare a vanilla baseline against **AGS** with Solution-Evo enabled across five languages (Python3, C++, Java, Go, Kotlin), using gemini-3-flash-preview as the backbone and a reflection budget of 3 rounds. We report three groups of metrics covering functional correctness, runtime and memory efficiency, and human-referenced competitiveness, with full definitions provided in Table 6 in the appendix.

Results and Analysis. Table 3 and Figure 3 reveal three key findings for self-evolving code agents under execution-guided evaluation and human-submission comparison. (i) **Self-evolution consistently improves functional correctness across languages.** Solution-Evo increases pass rates by 10.1–26.7% across the five languages, with the largest gain in Kotlin and high solved counts in compiled languages, including 99 problems in C++ and 98 in Java. Execution-blocking errors, including compile, runtime, and answer errors, are reduced to near zero, suggesting that inference-time feedback effectively repairs both format- and logic-level failures. (ii) **Execution-guided evolution improves efficiency beyond correctness.** Average runtime decreases in all languages, with a 7.8% reduction in Python3 and larger reductions of 19.8–46.4% in compiled languages. These gains align with fewer time-limit-exceeded errors, indicating that the agent not only fixes invalid outputs but also discovers more efficient algorithms. Memory usage decreases in most compiled languages by 4.8–16.6%, while increasing modestly in Python3 and Kotlin, likely due to auxiliary data structures introduced for correctness or speed. (iii) **Evolved solutions become more competitive relative to human submissions.** Runtime beats improve in compiled languages by up to 30.8%, while Python3 shows a modest decrease, consistent with a memory-speed trade-off. Memory beats improve in four of the five languages by 7.0–23.5%, but decrease in Kotlin, suggesting that long-tail languages may favor correctness over memory efficiency. Overall, Solution-Evo provides a strong default strategy for algorithmic coding tasks by combining execution feedback, iterative self-repair, and measurable competitiveness against human submissions.

6 Limitations and Impact Statement

First, self-evolution introduces additional inference rounds that increase latency and token consumption, and systematic analysis of the efficiency-effectiveness trade-off under strict budget constraints remains future work. Second, while **AGP** provides a unified protocol interface for all RSPL resource types, our experiments focus on Prompt-Evo, Solution-Evo, and Agent-Evo as primary comparison targets. Evolution of Environment and Memory resources has been implemented but not yet evaluated as independent ablation targets, and we leave this to future work. On the impact side, self-evolving agent systems may exhibit unintended behavioral drift if evolution objectives are misspecified or reward signals are noisy. The version control and rollback mechanisms in SEPL provide basic safeguards, but rigorous alignment verification remains an open challenge for broader deployment.

7 Conclusion

We presented **AGP**, a two-layer self-evolution protocol that decouples the evolutionary substrate from optimization logic, standardizing how agent resources are registered, versioned, and evolved. Instantiated as **AGS**, the protocol drives consistent improvements across scientific reasoning, open-ended agent tasks, and algorithmic code generation, demonstrating that a single evolution mechanism generalizes across task types and resource categories. We believe **AGP** offers a reusable foun-

dation for future work on multi-agent collaboration, safe online adaptation, and human-aligned self-improvement in dynamic real-world environments.

References

- [1] Anthropic. Equipping agents for the real world with agent skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Accessed October 2025.
- [2] Anthropic. Introduction to agent skills. <https://anthropic.skilljar.com/introduction-to-agent-skills>, October 2025.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2023.
- [6] Desearch-ai. desearch.py: Official Async Python SDK for the Desearch API. <https://github.com/Desearch-ai/desearch.py>, 2024.
- [7] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.
- [8] Google. Langfun: Object-Oriented Programming for Language Models. <https://github.com/google/langfun>, 2024.
- [9] Google. A2a: A new era of agent interoperability. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, April 2025. Google Developers Blog. Accessed: 2026-04-20.
- [10] H2O.ai. Enterprise h2oGPTe: Agentic AI for Generative and Predictive Intelligence. <https://h2o.ai/platform/enterprise-h2ogpte/>, 2025.
- [11] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- [12] Zhipeng Hou, Junyi Tang, and Yipeng Wang. Halo: Hierarchical autonomous logic-oriented orchestration for multi-agent llm systems. *arXiv preprint arXiv:2505.13516*, 2025.
- [13] J. Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.
- [14] Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.
- [15] HuggingFace. Open-source DeepResearch - Freeing Our Search Agents. <https://huggingface.co/blog/open-deep-research>, 2024.
- [16] LeetCode. Leetcode online judge. <https://leetcode.com>. Accessed 2025.
- [17] Jiarun Liu, Shiyue Xu, Shangkun Liu, Yang Li, Wen Liu, Min Liu, Xiaoqing Zhou, Hanmin Wang, Shilin Jia, Shaohua Tian, et al. Joyagent-jdgenie: Technical report on the gaia. *arXiv preprint arXiv:2510.00510*, 2025.

- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhunoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [19] Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.
- [20] NousResearch. Hermes Agent: A Self-Improving Open-Source Agent Framework. <https://github.com/NousResearch/hermes-agent>, 2025.
- [21] OpenAI. Introducing Deep Research. <https://openai.com/index/introducing-deep-research/>, February 2025.
- [22] openJiuwen. openJiuwen Agent Platform. <https://openjiuwen.com/en/>, 2026.
- [23] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [24] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- [25] Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 7957–7968, 2023.
- [26] Yujia Qin, Shihao Liang, Yining Ye, Kunliang Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- [27] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, et al. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. *arXiv preprint arXiv:2505.20286*, 2025.
- [28] David Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- [29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [32] Minjie Shen and Qikai Yang. From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent, 2025.
- [33] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [34] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, et al. Toolorchestra: Elevating intelligence via efficient model and tool orchestration. *arXiv preprint arXiv:2511.21689*, 2025.

- [35] Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. Evoagentx: An automated framework for evolving agentic workflows. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 643–655, 2025.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [37] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [39] Chengyue Yu, Siyuan Lu, Chenyi Zhuang, Dong Wang, Qintong Wu, Zongyue Li, Runsheng Gan, Chunfeng Wang, Siqi Hou, Gaochi Huang, et al. Aworld: Orchestrating the training recipe for agentic ai. *arXiv preprint arXiv:2508.20404*, 2025.
- [40] Mert Yuksekgonul, Federico Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025.
- [41] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
- [42] Hongwei Zhang, Ji Lu, Shiqing Jiang, Chenxiang Zhu, Li Xie, Chen Zhong, Haoran Chen, Yurui Zhu, Yongsheng Du, Yanqin Gao, Lingjun Huang, Baoli Wang, Fang Tan, and Peng Zou. Co-sight: Enhancing llm-based agents via conflict-aware meta-verification and trustworthy reasoning with structured facts. *arXiv preprint arXiv:2510.21557*, 2025.
- [43] Wentao Zhang, Ce Cui, Yilei Zhao, Yang Liu, and Bo An. Agentorchestra: A hierarchical multi-agent framework for general-purpose task solving. 2025.
- [44] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*, 2022.
- [45] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- [46] Zoom AI. HLE Leaderboard. <https://huggingface.co/spaces/zoom-ai/hle-leaderboard>, 2025.

A Notation

We summarize the main mathematical symbols and their meanings in Table 4. Symbols are organized into six functional categories (highlighted in grey) following the two-layer structure of **AGP**. The first three cover the **RSPL substrate**: indexing conventions, the resource entity tuple, and protocol-registered resources including the context manager $\mathcal{M}_\tau$ and server interface $\mathcal{A}_\tau$. The remaining three cover the **SEPL layer**: evolvable variables and the trainable subspace Θ , the auxiliary spaces ($\mathcal{P}$, $\mathcal{Z}$, $\mathcal{H}$, $\mathcal{D}$, $\mathcal{G}$, $\mathcal{S}$) and five canonical reflection operators $\{\rho, \sigma, \iota, \varepsilon, \kappa\}$, and iteration-level variables of the evolutionary loop.

Table 4: Notation used in the paper. Grey rows indicate categories.

Symbol	Description
Indexing and Sets	
$\mathcal{T}$	Set of RSPL entity types, {PROMPT, AGENT, TOOL, ENV, MEM}.
τ	Entity type index, $\tau \in \mathcal{T}$.
$\mathcal{I}_\tau$	Index set of resource instances of type τ .
i	Instance index, $i \in \mathcal{I}_\tau$.
$\mathbb{V}$	Space of version strings.
$\wp(\cdot)$	Power set operator.
RSPL Resource Entity (Def. E.1)	
$e_{\tau,i}$	Resource entity tuple $(n_{\tau,i}, d_{\tau,i}, \phi_{\tau,i}, g_{\tau,i}, m_{\tau,i})$.
$n_{\tau,i}$	Unique resource name.
$d_{\tau,i}$	Short description.
$\phi_{\tau,i} : \mathcal{X}_\tau \rightarrow \mathcal{Y}_\tau$	Input-to-output mapping of the resource.
$g_{\tau,i}$	Evolvability marker, $g_{\tau,i} \in \{0, 1\}$, indicating whether the resource is evolvable.
$m_{\tau,i}$	Auxiliary metadata dictionary.
$\mathcal{E}_\tau$	Set of resource entities of type τ .
RSPL Registration Record (Def. E.2)	
$c_{\tau,i}$	Registration record $(e_{\tau,i}, v_{\tau,i}, \eta_{\tau,i}, \theta_{\tau,i}, \mathcal{F}_{\tau,i})$.
$\mathcal{C}_\tau$	Set of registration records for type τ .
$v_{\tau,i}$	Version string of the resource instance.
$\eta_{\tau,i}$	Implementation descriptor (e.g., import path, class, or source).
$\theta_{\tau,i}$	Instantiation parameters (e.g., constructor arguments).
$\mathcal{F}_{\tau,i}$	Exported representations for LLM interaction (schemas/text/structured args).
Protocol-registered Resource (Def. E.3)	
$\mathcal{R}_\tau$	Type-specific registry of protocol-registered resources.
$\mathcal{R}$	Global registry, $\bigcup_\tau \mathcal{R}_\tau$.
$\mathcal{M}_\tau$	Context manager for type τ (maintains registry and version lineage).
$\mathcal{A}_\tau$	Server-exposed interface for type τ (delegates to $\mathcal{M}_\tau$).
r_τ	Type-level registered resource triple $(\mathcal{C}_\tau, \mathcal{M}_\tau, \mathcal{A}_\tau)$.
SEPL Variables, Spaces, and Operators	
$\mathcal{V}_{\text{evo}}$	Universal set of evolvable variables (all managed entities plus execution artifacts).
v	A variable in $\mathcal{V}_{\text{evo}}$.
g_v	Learnability constraint for variable v (binary).
Θ	Trainable subspace, $\{v \in \mathcal{V}_{\text{evo}} \mid g_v = 1\}$.
y	Execution artifacts (e.g., outputs and reasoning traces).
$\mathcal{P}$	Message space carrying auxiliary signals (traces, hypotheses, gradients, rewards) between operators.
$\mathcal{P}_{\text{in}}, \mathcal{P}_{\text{out}}$	Incoming and outgoing message types of a SEPL operator.
f	A SEPL operator, $f : \mathcal{V}_{\text{evo}} \times \mathcal{P}_{\text{in}} \rightarrow \mathcal{V}'_{\text{evo}} \times \mathcal{P}_{\text{out}}$.
$\mathcal{Z}$	Trace space (execution observations).
$\mathcal{H}$	Hypothesis space (causal failure attributions).
$\mathcal{D}$	Modification space (proposed resource changes).
$\mathcal{G}$	Objective specification (task goals and safety invariants).
$\mathcal{S}$	Evaluation space (performance metrics and safety status).
$\rho, \sigma, \iota, \varepsilon, \kappa$	Reflect, Select, Improve, Evaluate, and Commit operators (reflection instantiation).
Optimization Loop (Alg. 1)	
A	Agentic system.
T	Optimization budget (number of iterations).
t	Iteration index.
$\mathcal{V}_{\text{evo}}^{(t)}$	Evolvable state at iteration t .
$\mathcal{P}^{(t)}$	Message passed between operators at iteration t .
$\mathcal{Z}^{(t)}$	Observational trace at iteration t .
$\mathcal{H}^{(t)}$	Hypotheses at iteration t .
$\mathcal{D}^{(t)}$	Proposed modifications at iteration t .
$\tilde{\mathcal{V}}_{\text{evo}}^{(t+1)}$	Candidate state after applying modifications.
$\mathcal{S}^{(t+1)}$	Evaluation result for the candidate state.

B Code, Prompts, and Resources

All source code, agent prompts, and optimizer prompts for **AGP** are organized as follows.

Agent and Optimizer Prompts. All system prompts and task-specific prompts used by the agents, as well as the prompts used by each optimizer instantiation (Reflection Optimizer, TextGrad, Reinforce++, and GRPO), are provided in the `autogenesis/` directory of the supplementary material. The directory is structured by component: each subdirectory corresponds to a distinct agent role or optimizer module and contains the associated prompt templates.

Self-Evolving Code Agent Benchmark Data. The benchmark problems, test cases, and reference solutions for the Self-Evolving Code Agent Benchmark are provided in the `data/` directory of the supplementary material. The dataset covers all collected LeetCode-derived problems across five programming languages (Python, C++, Java, JavaScript, and Go). The evaluation scripts are located in `autogenesis/src/benchmark/`.

C Comparison with Other Protocols

Table 5 provides a structured protocol-level comparison between **AGP**, Google A2A [9], and Anthropic MCP [1]. While A2A and MCP have standardized inter-agent communication and model-to-tool invocation respectively, both operate solely at the level of message passing and invocation, leaving internal resource states opaque and providing no primitives for lifecycle management, version lineage, or controlled state mutation. These are precisely the three properties (**Decoupling**, **Safety & Auditability**, and **Formalism**) that **AGP** is designed to provide. The comparison is organized into five dimensions (grey rows), with blue-highlighted entries marking capabilities that are prerequisite for closed-loop self-evolution but absent from communication- or invocation-centric protocols.

Table 5: Protocol-level comparison: **Autogenesis Protocol (AGP)** vs. Google A2A vs. Anthropic MCP across key dimensions for agentic systems and self-evolution. Symbols: ✓ = Supported, △ = Partial, ✗ = Not supported. Highlighted rows (blue background) emphasize evolution-enabling capabilities.

Dimension	AGP	A2A	MCP
<i>Basic Information</i>			
Proposer	Our work	Google	Anthropic
Protocol Focus	Self-evolution Agentic System	Multi-agent System Collaboration	Tool
Entity Scope	Prompt/Agent/Tool/Env/Memory	Agent/Tool	Tool
<i>Agent and System Capabilities</i>			
Agent First-Class	✓	✓	✗
Multi-Agent	✓	✓	✗
Tracer	✓	△	✗
Memory as Resource	✓	✗	✗
<i>Evolvable Resource Management</i>			
Lifecycle Ops	✓	△	✗
Versioning and Rollback	✓	✗	✗
Registry and Retrieval	✓	△	△
Contract Generation	✓	△	✗
<i>Self-Evolution Mechanism</i>			
Closed-Loop Evolution	✓	✗	✗
Operatorized Updates	✓	✗	✗
Auditability	✓	△	△
<i>General and Ecosystem</i>			
Model-Agnostic	✓	✓	✓
Scalability	$O(\log n)$	$O(n^2)$	$O(n)$
Open Ecosystem	✓	△	△

C.1 Basic Information

Proposer. Google’s A2A [9] is introduced as a protocol for multi-agent communication, enabling agents to collaborate through standardized interaction primitives. Anthropic’s MCP [1] standardizes model to tool invocation interfaces. In contrast, **AGP** is proposed in this work as a self-evolution protocol for composable, auditable, and safely updateable agentic systems.

Protocol Focus. **AGP** focuses on closed-loop improvement of agentic systems by organizing resource updates through typed protocol operators and versioned state transitions. A2A primarily addresses inter-agent communication and task delegation. MCP primarily addresses standardized model to tool invocation.

Entity Scope. **AGP** governs heterogeneous entities, including prompts, agents, tools, environments, and memory, as protocol-registered resources with explicit state and version lineage. This design supports component-level evolution, including prompt refinement and tool code updates. A2A treats agents and tools as interaction endpoints without unified lifecycle management. MCP exposes tools as callable interfaces but does not model them as evolvable components with version lineage.

C.2 Agent and System Capabilities

Agent First-Class. **AGP** models agents as managed protocol components with explicit schemas, metadata, and lifecycle hooks. This enables registration, discovery, orchestration, and controlled updates. A2A is agent-centric but treats agents primarily as service endpoints without unified lifecycle management or version lineage. MCP does not define agents as protocol components and instead focuses on model to tool connectivity.

Multi-Agent. **AGP** supports multi-agent configurations as part of its system substrate, enabling coordinated execution with traceability and evolution-ready state. A2A directly supports agent-to-agent collaboration. MCP does not treat multi-agent orchestration as a protocol-level concern.

Execution Tracing. **AGP** provides protocol-level trace capture over inputs, outputs, intermediate decisions, and tool calls. These traces provide the learning signals required for auditable evolution. A2A and MCP leave tracing to application-level instrumentation, which can lead to inconsistent observability across deployments.

Memory as Resource. **AGP** models memory as a first-class protocol resource with explicit read and write interfaces, state, and version lineage. This enables persistent cross-task improvement and reproducible evolution. A2A and MCP do not prescribe a memory management protocol and instead delegate persistence to external systems.

C.3 Evolvable Resource Management

Lifecycle Ops. **AGP** provides standardized lifecycle operators for initialization, registration, construction, and decommissioning. These operators ensure that updates are applied to well-defined and protocol-governed targets. A2A offers partial lifecycle support for agents. MCP does not define lifecycle management across heterogeneous component types.

Versioning and Rollback. Version lineage and rollback form the safety foundation of closed-loop evolution. Each update produces an immutable snapshot that supports comparison, auditing, and restoration after regressions. **AGP** integrates versioning as a first-class protocol capability. A2A and MCP do not natively support version lineage over protocol-managed components, which limits systematic evolution.

Registry and Retrieval. **AGP** maintains a unified registry of protocol-registered resources and supports semantic retrieval to reduce duplication and improve composability across tasks. A2A and MCP provide partial discovery mechanisms, but they do not define a unified management plane over heterogeneous component types.

Contract Generation. **AGP** supports automated generation of consolidated capability specifications that enumerate tool actions, arguments, preconditions, and usage constraints. This provides a principled form of context engineering that reduces prompt bloat and improves orchestration reliability. A2A and MCP rely on static descriptions or application-layer documentation without protocol-level contract aggregation.

C.4 Self-Evolution Mechanism

Closed-Loop Evolution. **AGP** is built around an iterative improvement loop consisting of execution, reflection, proposal generation, evaluation, and commitment. This loop enables sustained and evidence-grounded refinement rather than one-off adaptation. A2A and MCP do not provide native self-evolution primitives.

Operatorized Updates. **AGP** expresses state mutations as typed and composable SEPL operators with well-defined input and output contracts. This enables controlled and repeatable evolution. A2A and MCP do not define a composable operator interface for resource modification, leaving updates to application-specific logic.

Auditability. **AGP** enforces auditability at the protocol level by recording each state transition, the execution evidence that motivated it, and the evaluation outcome that justified it. This audit trail is supported by version lineage and rollback. A2A and MCP provide only partial audit trails through external instrumentation and do not offer protocol-level guarantees.

C.5 General and Ecosystem

Model-Agnostic. This dimension assesses whether a protocol can operate across different LLM backends and providers. **AGP** is model-agnostic by design through a unified model interface layer. A2A and MCP are also broadly model-agnostic because they define interaction standards rather than binding the protocol to a specific model.

Scalability. Scalability characterizes how coordination and discovery behave as the number of components increases. **AGP** supports scalable management by treating heterogeneous components as registry-governed resources with retrieval mechanisms, enabling efficient lookup and controlled orchestration. A2A may incur increasing coordination overhead as interactions become denser in large multi-agent settings. MCP standardizes tool interfaces but may still require application-level orchestration for large tool or resource sets.

Open Ecosystem. Open ecosystem support refers to whether a protocol can enable reusable and interoperable components. **AGP** provides a protocol stack for managing, evolving, and auditing agentic components, which supports component sharing and safe integration. A2A and MCP provide partial ecosystem support through interoperability and tool interface standardization, but they typically require additional layers for evolution-ready resource management.

D Details of the Self-Evolving Code Agent Benchmark

D.1 Benchmark Design Rationale

Our benchmark is designed to evaluate self-evolving code agents under execution-grounded and human-referenced conditions. Unlike conventional code generation benchmarks that primarily assess final correctness, self-evolving agents can improve within a single inference episode by producing an initial solution, observing execution feedback, reflecting on failure modes, and revising the solution accordingly. This adaptive process requires a benchmark that measures not only whether the final submission is accepted, but also how performance evolves throughout refinement. Accordingly, our benchmark is motivated by three objectives: (i) evaluating inference-time self-evolution on executable code, (ii) calibrating agent performance against human submission distributions, and (iii) assessing cross-language robustness under long-tail language usage.

The first objective is to make self-evolution directly measurable during inference. In algorithmic coding tasks, execution feedback provides concrete and fine-grained signals, including compilation status, runtime errors, wrong answers, time-limit violations, memory-limit violations, and execution statistics for accepted submissions. These signals allow the agent to identify whether a failure stems from syntax errors, interface mismatches, corner-case logic, algorithmic inefficiency, or excessive resource usage. A benchmark for self-evolving agents should therefore expose such feedback at each refinement round and record the resulting improvement trajectory. This design distinguishes agents that solve problems through stable and efficient refinement from those that achieve correctness only through costly or unstable trial-and-error behavior.

The second objective is to evaluate coding performance relative to human submissions. Absolute pass rates are informative but insufficient for assessing practical coding competence, since they do not indicate whether an accepted solution is efficient compared with human-written solutions. We therefore build on the LeetCode online judge, which reports runtime and memory usage for accepted submissions, together with percentile-based *runtime beats* and *memory beats* statistics computed from human submission distributions. These human-referenced metrics provide an interpretable basis for assessing whether self-evolution improves not only correctness, but also competitiveness relative to human programmers.

The third objective is to evaluate robustness across programming languages, including long-tail languages. Many coding benchmarks are dominated by Python or other high-resource languages, which can obscure language-specific failures related to syntax, libraries, typing discipline, compilation, and runtime behavior. LeetCode provides standardized starter code across a broad set of languages, enabling the same problem to be evaluated under comparable interfaces in Python3, C++, Java, Go, and Kotlin. This design supports systematic analysis of whether self-evolution generalizes across languages and whether feedback-driven refinement remains effective across both high-resource and lower-resource programming ecosystems.

Overall, the benchmark provides a controlled setting for evaluating self-evolving code agents as dynamic problem solvers. By combining execution-based judging, iterative feedback, human-referenced efficiency statistics, and multi-language evaluation, it jointly measures functional correctness, resource efficiency, refinement dynamics, and human-relative competitiveness under a unified protocol.

D.2 Benchmark Construction

Data Collection. We collect the full set of 3,822 programming problems available on LeetCode at the time of crawling. For each problem, we extract the natural-language statement, official input and output examples, constraints, platform-provided difficulty label, topical tags, and language-specific starter code templates. The topical tags characterize the algorithmic concepts required by each problem, including arrays, trees, graphs, dynamic programming, greedy methods, binary search, and mathematics. These annotations support stratified analysis across difficulty levels, algorithmic categories, and programming languages. Figure 5 summarizes the tag and difficulty distributions of the selected problems.

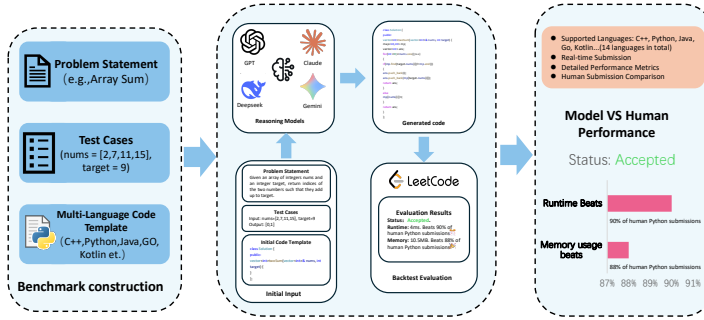


Figure 4: Self-evolving code agent benchmark evaluation pipeline. Figure 5: Problem distribution.

The collected data are normalized into a unified problem representation. Each instance contains a fixed task specification, official examples, a language-specific starter template, and metadata for difficulty and topic categories. We preserve the original interface required by the online judge so that generated code can be submitted without modifying function signatures or class definitions. This design ensures that performance differences arise from agent behavior rather than inconsistencies in task formatting or evaluation interfaces. We conduct quality checks by filtering malformed records, removing duplicates, and verifying that starter templates are available for all target languages. From the full pool, we select 100 recently released problems to mitigate training-data contamination. The selected problems span diverse topical categories and difficulty levels, and are instantiated across Python3, C++, Java, Go, and Kotlin to enable controlled cross-language evaluation.

Problem Characteristics. LeetCode-style algorithmic problems provide a controlled and challenging setting for evaluating code-agent competence. Each task specifies explicit constraints and precise input and output behavior, requiring instruction following, edge-case coverage, and faithful implementation under a fixed interface. The breadth of tags and difficulty levels evaluates algorithm selection, data-structure proficiency, and complexity-aware reasoning. Because evaluation is execution-based, brittle solutions can be exposed through concrete failures such as off-by-one errors, corner-case bugs, interface mismatches, and language-specific pitfalls. Standardized starter templates across languages further enable systematic cross-language comparison, including robustness analysis in long-tail languages. Since a solution can be revised within the same inference episode, this setting directly measures agentic capabilities such as self-repair, feedback-grounded hypothesis testing, and efficiency-aware optimization under runtime and memory constraints.

Problem Evaluation. For each problem, the agent receives a fixed input representation and submits generated code to the official execution-based judge, which evaluates functional correctness on hidden test cases and reports resource usage statistics. This protocol ensures all agents are assessed under identical task inputs, execution conditions, and scoring criteria. When evaluating agents with self-evolution capability, the agent is additionally allowed to iteratively refine its solution within the same problem-solving episode under a fixed budget of 3 rounds, using execution feedback from the online judge to reflect on failure causes, identify actionable error patterns, and propose targeted code revisions, while keeping the task specification, prompt schema, and evaluation interface unchanged.

D.3 Evaluation Metrics

Table 6: Evaluation metrics for the algorithmic coding benchmark.

Metric	Description
Capability metrics	
PR	Number of problems passing all hidden test cases within time and memory limits.
TLE	Number of problems exceeding the allowed execution time limit.
MLE	Number of problems exceeding the allowed memory usage.
CE	Number of problems where generated code failed to compile.
RE	Number of problems encountering a runtime error during execution.
WA	Number of problems producing incorrect output.
TO	Number of problems where the model failed to respond within the timeout.
RpE	Number of problems where the model returned an invalid or unparseable response.
Efficiency metrics	
AR	Mean runtime in milliseconds over accepted solutions.
AM	Mean memory usage in megabytes over accepted solutions.
APC	Mean number of test cases passed before failure.
Human-referenced metrics	
ARB	Percentage of accepted solutions whose runtime outperforms human submissions.
AMB	Percentage of accepted solutions whose memory usage outperforms human submissions.

We report three groups of metrics that capture complementary aspects of code-agent performance. Capability metrics evaluate functional correctness and diagnose execution-blocking failure modes. PR measures the number of fully accepted problems, while TLE, MLE, CE, RE, WA, TO, and RpE identify whether failures arise from algorithmic inefficiency, excessive memory use, compilation errors, runtime exceptions, incorrect logic, response timeout, or invalid output formatting. These metrics are particularly important for self-evolving agents because different failure modes correspond to different refinement opportunities.

Efficiency metrics characterize the computational quality of accepted solutions and the progress made by partially correct submissions. AR and AM summarize runtime and memory usage over accepted solutions, which allows us to assess whether self-evolution improves efficiency rather than merely increasing pass rate. APC measures how many test cases are passed before failure and provides a fine-grained signal for unsuccessful submissions. This metric is useful when an agent progresses from early failure to passing most hidden tests, even if the final solution is not accepted.

Human-referenced metrics situate accepted agent solutions within the empirical distribution of human submissions. ARB measures the fraction of accepted human submissions whose runtime is slower than the agent solution, while AMB measures the analogous fraction for memory usage. These

metrics provide an interpretable basis for comparing evolved agent solutions with human-written solutions and help determine whether an accepted solution is merely correct or also competitive.

For self-evolving agents, all metrics can be computed at each refinement round as well as for the final submission. This enables trajectory-level evaluation of inference-time improvement, including whether correctness increases across rounds, whether runtime and memory usage improve or degrade, and whether human-relative competitiveness changes after reflection and revision. The benchmark therefore evaluates both endpoint performance and the refinement process through which an agent reaches that endpoint.

E Details of Self-Evolution Protocol

E.1 Design Motivation

Existing LLM-based agent systems pursue self-improvement in largely ad hoc ways: prompts, tools, and memory are tightly coupled to agent logic, updated without version control, and impossible to roll back when an update degrades performance. This architecture fragility motivates the two-layer design of **AGP**. We outline the core motivating principles below.

- **Decoupling substrate from logic.** In most existing frameworks, what an agent operates over (prompts, tools, memory) and how it evolves them (optimization algorithms, feedback loops) are interleaved in a single codebase. This coupling makes it difficult to swap, reuse, or safely update individual components. **AGP** separates the *evolvable substrate* (RSPL) from the *evolution logic* (SEPL), so that any compliant optimizer can be applied to any registered resource without modifying the component itself. This modularity is essential for principled, reproducible self-evolution.
- **Safety and auditability through lifecycle management.** Self-evolving agents modify their own components at runtime, which introduces risks of cascading failures or undetectable regressions. Without explicit version control and rollback support, a single bad update can silently degrade system behavior. RSPL endows every resource with versioned state and a controlled mutation interface, ensuring that each evolutionary step is traceable, reversible, and subject to explicit commit or rollback decisions. SEPL enforces that updates proceed only after formal evaluation, making every change auditable by design.
- **Formalism over heuristics.** Prior self-evolution approaches apply modifications heuristically, for example, by prompting a model to "improve itself" or by directly patching code, with no standardized interface governing what constitutes a valid update cycle. This informality makes it impossible to guarantee safety or reason about correctness across runs. SEPL formalizes the update cycle as a closed-loop operator interface, transforming ad hoc modifications into a rigorous protocol with well-defined pre- and post-conditions. This formalism enables algorithm-agnostic instantiation: the same operator interface supports prompt optimization, reinforcement learning, and gradient-free search.
- **Uniform abstraction across heterogeneous components.** Agent systems compose heterogeneous entities, including LLM instructions, external tool scripts, MCP services, and in-context memory, that are typically managed through disparate, component-specific mechanisms. **AGP** provides a single, unified resource entity abstraction that encompasses all five types (Prompt, Agent, Tool, Environment, Memory), enabling SEPL to apply the same evolution operators uniformly across all components without special-casing.

Together, these principles ground the two-layer architecture in a coherent design philosophy: RSPL provides a stable, typed, versioned substrate that renders agent internals observable and controllable, while SEPL provides a safe, formal operator interface that governs how those internals are updated. The remainder of this section specifies each layer in detail.

E.2 Layer 1: Resource Substrate Protocol Layer

The Resource Substrate Protocol Layer (RSPL) defines the evolvable substrate as a set of protocol-registered resources with explicit state, lifecycle, and version lineage. In this paper, these resources comprise (i) *instructions* (Prompt), (ii) *decision policies* (Agent), (iii) *actuation interfaces* (Tool), which encompass native tool scripts, MCP tools [1], and agent skills [2], (iv) *task/world dynamics*

(Environment), and (v) *persistent state* (Memory). Crucially, resources in RSPL are *passive*: they encapsulate no optimization logic and cannot self-modify; all observations and state transitions occur only through controlled, interface-mediated operations invoked by higher layers.

E.2.1 Core Entities

We focus on these five entity types as a minimal yet expressive substrate for agentic systems. This choice is not intended to be exhaustive, but rather to identify a common denominator across modern agent stacks and provide a uniform target space on which SEPL can operate.

Definition E.1 (Resource Entity). A resource entity of type τ and its type-level collection can be represented as:

$$\begin{aligned} e_{\tau,i} &= (n_{\tau,i}, d_{\tau,i}, \phi_{\tau,i}, g_{\tau,i}, m_{\tau,i}), \\ \mathcal{E}_{\tau} &= \{e_{\tau,i} \mid i \in \mathcal{I}_{\tau}\}, \end{aligned} \quad (6)$$

where $\mathcal{T} = \{\text{PROMPT}, \text{AGENT}, \text{TOOL}, \text{ENV}, \text{MEM}\}$ denotes the set of RSPL entity types, $\tau \in \mathcal{T}$ indexes the entity type, $\mathcal{I}_{\tau}$ is the index set of resource instances of type τ , and $i \in \mathcal{I}_{\tau}$ indexes an individual instance. Here $n_{\tau,i}$ is a unique resource name, $d_{\tau,i}$ is a short description, $\phi_{\tau,i} : \mathcal{X}_{\tau} \rightarrow \mathcal{Y}_{\tau}$ is an input-to-output mapping, $g_{\tau,i} \in \{0, 1\}$ is an evolvability marker, and $m_{\tau,i}$ is an auxiliary metadata dictionary.

A key motivation for making prompt, tool, and memory explicit RSPL resources is *decoupling*. Many agent systems package prompts, tools, and memory as internal components of an agent, which entangles agent logic with task-specific instructions and capability bundles, increasing maintenance and limiting transfer. By externalizing them as first-class, versioned resources with standardized interfaces, the same tool-calling agent policy can be paired with different prompts and tool sets, and deployed unchanged across tasks and environments.

To support resource registration, unified management, and instantiation, RSPL stores a serializable registration record for each resource instance.

Definition E.2 (Resource Registration Record). A resource registration record and its type-level collection can be represented as:

$$\begin{aligned} c_{\tau,i} &= (e_{\tau,i}, v_{\tau,i}, \eta_{\tau,i}, \theta_{\tau,i}, \mathcal{F}_{\tau,i}), \\ \mathcal{C}_{\tau} &= \{c_{\tau,i} \mid i \in \mathcal{I}_{\tau}\}, \end{aligned} \quad (7)$$

where $\tau \in \mathcal{T}$ indexes the entity type and $i \in \mathcal{I}_{\tau}$ indexes an individual instance. Here $e_{\tau,i}$ is the resource entity tuple defined in Theorem E.1, $v_{\tau,i} \in \mathbb{V}$ is a version string, $\eta_{\tau,i}$ is an implementation descriptor (e.g., import path, class definition, or source-code string), $\theta_{\tau,i}$ are instantiation parameters (e.g., constructor arguments), and $\mathcal{F}_{\tau,i}$ is a set of exported representations used by LLMs to interact with the resource (e.g., function-calling schema, plain text, and structured argument schema).

Definition E.3 (Protocol-registered resource). For each entity type τ , let $\mathcal{R}_{\tau}$ denote the type-specific registry of protocol-registered resources, and let $\mathcal{R} = \bigcup_{\tau} \mathcal{R}_{\tau}$ denote the global registry. RSPL binds each entity type τ to a dedicated context manager $\mathcal{M}_{\tau}$ and a server-exposed interface $\mathcal{A}_{\tau}$. We represent the type-level registered resource as

$$r_{\tau} = (\mathcal{C}_{\tau}, \mathcal{M}_{\tau}, \mathcal{A}_{\tau}), \quad (8)$$

where each $c_{\tau,i} \in \mathcal{C}_{\tau}$ is a registration record in Theorem E.2. The context manager $\mathcal{M}_{\tau}$ maintains the collection $\mathcal{C}_{\tau}$, the version lineage for type τ , and implements lifecycle and update operations over these records; the server-exposed interface $\mathcal{A}_{\tau}$ encapsulates $\mathcal{M}_{\tau}$ and exposes a unified external interface by delegating requests to the corresponding context-manager routines.

E.2.2 Context Manager

The context manager implements the management plane for each resource type. Beyond lifecycle control and dependency constraints, it maintains (i) an active registry of materialized resources and (ii) a versioned history for restoration. Its exported API exposes operators for lifecycle (`init`, `build`), retrieval (`list`, `get`), versioning (`update`, `restore`), execution (`run`), and serialization (`save_to_json`, `load_from_json`, `save_contract`, `load_contract`). The manager explicitly supports *contract generation*, producing a consolidated capability and constraint specification for the managed entities, which provides stable, up-to-date descriptions that improve reliability and reduce

prompt bloat, enabling systematic *context engineering* via controlled prompt injection. For instance, for tools (which may be native tool scripts, MCP-connected tools [1], or agent skills) the contract can take a `skills.md`-style form [2] that enumerates tool actions, arguments, preconditions, and usage constraints. The exported management interface implemented by $\mathcal{M}_\tau$ and exposed by $\mathcal{A}_\tau$ are as follows:

Table 7: Operator set of Context Manager and Server Interface.

Operator	Description
Lifecycle & Registration	
<code>init</code>	Auto discover resources and register the resource configuration to the registry.
<code>build</code>	Build a resource instance from code and configuration.
<code>register</code>	Register a new resource instance with a unique name and version.
<code>unregister</code>	Unregister a resource instance from the active registry and version history.
Retrieval & Inspection	
<code>get</code>	Retrieve a resource instance by name from the active registry.
<code>get_info</code>	Retrieve a resource configuration by name from the active registry.
<code>list</code>	List all registered resource names.
<code>retrieve</code>	Retrieve similar resources via semantic search when supported.
<code>get_state</code>	Get the current state of a resource instance when supported.
Versioning	
<code>update</code>	Update a resource implementation and generate a new version.
<code>copy</code>	Duplicate a resource with an optional new name and version.
<code>restore</code>	Restore a specific historical version by name and version string.
<code>get_variables</code>	Expose resource code/configuration as evolvable variables.
<code>set_variables</code>	Update resource variables and generate a new version.
Execution	
<code>run</code>	Run a resource instance with structured input.
Serialization	
<code>save_to_json</code>	Serialize configurations and version history to a JSON file.
<code>load_from_json</code>	Deserialize configurations and version history from a JSON file.
<code>save_contract</code>	Save the contract of a resource instance to a file.
<code>load_contract</code>	Load the contract of a resource instance from a file.

E.2.3 Server Interface

The server is introduced to encapsulate the context manager’s internal complexity and present a stable, simplified interface for external callers. It packages heterogeneous management routines behind a uniform set of endpoints with consistent request/response semantics, while delegating the implementation details to the context manager. This separation isolates clients from internal design changes, reduces coupling, and provides a single control plane through which the protocol mediates safe, version-aware interactions with RSPL resources.

E.2.4 Infrastructure Services

RSPL further includes cross-cutting services that support reliable evolution, including reproducibility, safe deployment, and versioned recovery:

Model manager. A unified model-API layer that standardizes calls across providers (e.g., OpenAI, Anthropic, Google, and OpenRouter, etc.), while supporting routing, fallback, and cost-aware selection to keep model access consistent as components evolve.

Version manager. Maintains version lineage for each resource, enabling rollback, branching, and diffing. Versions are auto-incremented identifiers (e.g., semantic versions) assigned on register or update, each referencing an immutable snapshot of the configuration record and associated artifacts for auditability and reproducibility.

Dynamic manager. Handles serialization and deserialization of resource configurations for persistence and transfer, enabling safe hot-swapping of resource configurations at runtime without restarting the agent system.

Trace manager. Captures fine-grained execution traces (inputs, outputs, intermediate decisions, tool interactions, etc.) for interpretability and debugging, and as training signals for dataset synthesis and retrospective improvement.

E.3 Layer 2: Self-Evolution Protocol Layer

The Self-Evolution Protocol Layer (SEPL) formalizes agentic system evolution as a generalized optimization problem over a heterogeneous state space, modeling evolutionary dynamics as a state transition function governed by a strictly typed operator algebra. By mediating all state mutations through standardized RSPL interfaces, SEPL guarantees that evolution is traceable, reversible, and safe-by-construction. While this paper focuses on the reflection-driven optimizer as the primary instantiation, the same state manipulation primitives also accommodate textual-gradient methods such as TextGrad [40] and reinforcement learning approaches such as GRPO [31] and Reinforce++ [14].

E.3.1 Evolvable Variables

To transition from heuristic adaptation to a systematic evolution protocol, SEPL introduces the concept of *variable lifting*: projecting discrete, heterogeneous RSPL resources (e.g., tool code, system prompts, memory modules, and environment configurations) onto a unified representation of evolvable variables. This homogenizes the interaction surface for all evolutionary operators and rigorously delineates the trainable subspace via an explicit learnability mask.

Definition E.4 (Evolvable Variable Set). We define the universal set of evolvable variables as

$$\mathcal{V}_{\text{evo}} = \left(\bigcup_{\tau \in \mathcal{T}} \mathcal{E}_{\tau} \right) \cup \{y\}, \quad (9)$$

where $\mathcal{E}_{\tau}$ denotes the set of resource entities of type τ governed by RSPL, and y encapsulates execution artifacts (final outputs and reasoning traces) that constitute the observational basis for retrospective optimization. Each variable $v \in \mathcal{V}_{\text{evo}}$ is associated with a binary learnability constraint $g_v \in \{0, 1\}$, strictly defining the trainable parameter subspace

$$\Theta = \{v \in \mathcal{V}_{\text{evo}} \mid g_v = 1\}. \quad (10)$$

The evolvability marker g_v allows SEPL to operate selectively: frozen components (e.g., a fixed tool API) are excluded from the trainable subspace, while designated evolvable resources (e.g., system prompts, tool implementations) are exposed for modification. This explicit masking ensures that only intended components are mutated during evolution.

E.3.2 Operator Algebra

Definition E.5 (SEPL Operator). Let $\mathcal{V}_{\text{evo}}$ be the evolvable variable set and $\mathcal{P}$ a *message space* carrying auxiliary signals (e.g., traces, hypotheses, gradients, or reward signals) passed between operators. A *SEPL operator* is a function

$$f : \mathcal{V}_{\text{evo}} \times \mathcal{P}_{\text{in}} \rightarrow \mathcal{V}'_{\text{evo}} \times \mathcal{P}_{\text{out}}, \quad (11)$$

where $\mathcal{P}_{\text{in}}, \mathcal{P}_{\text{out}} \subseteq \mathcal{P}$ are the incoming and outgoing message types, and $\mathcal{V}'_{\text{evo}}$ is the updated evolvable state. Operators are *composable*: the output $(\mathcal{V}'_{\text{evo}}, \mathcal{P}_{\text{out}})$ of one operator serves as the input to the next, enabling the construction of an evolutionary pipeline $f_n \circ \dots \circ f_1$. All mutations to $\mathcal{V}_{\text{evo}}$ must be routed through RSPL interfaces, ensuring every state transition is versioned, auditable, and reversible regardless of the specific optimizer instantiation.

The auxiliary spaces used by operators are: trace space $\mathcal{Z}$ (execution observations), hypothesis space $\mathcal{H}$ (causal failure attributions), modification space $\mathcal{D}$ (proposed resource changes), objective specification $\mathcal{G}$ (task goals and safety invariants), and evaluation space $\mathcal{S}$ (performance metrics and safety status). The five canonical operators of the reflection instantiation are $\{\rho, \sigma, \iota, \varepsilon, \kappa\}$, corresponding to REFLECT, SELECT, IMPROVE, EVALUATE, and COMMIT, operating over these spaces in sequence. Other instantiations (TextGrad, GRPO, Reinforce++) reuse the same operator interface but replace the internal logic of individual operators, as detailed in the method-specific subsections below.

E.3.3 Evolutionary Loop

Given an initial evolvable state $\mathcal{V}_{\text{evo}}^{(0)}$ and an empty message $\mathcal{P}^{(0)} = \emptyset$, the evolutionary loop at each iteration t applies a sequence of operators $f_1, \dots, f_n$ in composition:

$$(\mathcal{V}_{\text{evo}}^{(t+1)}, \mathcal{P}^{(t+1)}) = (f_n \circ \dots \circ f_1)(\mathcal{V}_{\text{evo}}^{(t)}, \mathcal{P}^{(t)}), \quad (12)$$

where each f_i reads the current state and incoming messages, produces an updated state and outgoing messages consumed by f_{i+1} . The loop repeats until convergence or budget exhaustion. By routing all state mutations through RSPL interfaces, each transition is versioned and reversible, guaranteeing that evolution is *grounded* in execution data, *traceable* through versioned updates, and *safe-by-construction*.

The specific operator sequence instantiated by each method determines the behavior of the loop. The reflection optimizer instantiates this loop with five operators: REFLECT maps execution traces and current state to causal failure hypotheses, SELECT identifies target evolvable entities and generates concrete modification proposals, IMPROVE applies proposals via RSPL interfaces to yield a candidate state, EVALUATE scores the candidate against the objective and safety invariants, and COMMIT conditionally accepts or rolls back the transition. TextGrad, GRPO, and Reinforce++ reuse the same loop structure but replace the internal logic of individual operators, as detailed in the method-specific subsections below.

E.3.4 Reflection Optimizer

Evolvable Variables. In the reflection-driven instantiation, the evolvable state is given by the lifted variable set $\mathcal{V}_{\text{evo}}$ introduced above. Concretely, $\mathcal{V}_{\text{evo}}$ includes RSPL-managed resources (e.g., prompts, tools, memories, and agent components) together with execution artifacts (e.g., the produced answer and reasoning trace). A binary learnability mask specifies which variables may be modified, allowing the optimizer to target only authorized components while keeping non-learnable resources fixed.

Operator Algebra. We instantiate SEPL with the canonical reflection-driven operator suite. The operator signatures and their intended roles are as follows.

- **Reflect** (ρ). Defined as $\rho : \mathcal{Z} \times \mathcal{V}_{\text{evo}} \rightarrow \wp(\mathcal{H})$, this operator bridges the gap between raw observation and optimization direction. It approximates the “semantic gradient” of the system by mapping high-dimensional execution traces to specific, causal failure hypotheses within the variable space.
- **Select** (σ). Formulated as $\sigma : \mathcal{V}_{\text{evo}} \times \wp(\mathcal{H}) \rightarrow \wp(\mathcal{D})$, this operator acts as the targeting policy. It identifies which evolvable entities within $\mathcal{V}_{\text{evo}}$ are implicated by the diagnostic hypotheses, then generates concrete modification proposals $\mathcal{D}$ targeting those entities, subject to structural constraints.
- **Improve** (ι). The mutation operator, $\iota : \mathcal{V}_{\text{evo}} \times \wp(\mathcal{D}) \rightarrow \mathcal{V}'_{\text{evo}}$, executes the physical state transition. It applies discrete updates $\mathcal{D}$ via standardized RSPL interfaces to yield a provisional candidate state.
- **Evaluate** (ε). Specified as $\varepsilon : \mathcal{V}'_{\text{evo}} \times \mathcal{G} \rightarrow \mathcal{S}$, this operator serves as the objective function. It maps the candidate state and goal specification to the evaluation space $\mathcal{S}$ (comprising quantitative scores and strict safety invariants).
- **Commit** (κ). Operating as $\kappa : \mathcal{V}'_{\text{evo}} \times \mathcal{S} \rightarrow \mathcal{V}_{\text{evo}}$, this function acts as a conditional gating mechanism. It utilizes the evaluation signals in $\mathcal{S}$ to govern state transition, rigorously enforcing safety invariants and performance monotonicity by accepting the candidate $\mathcal{V}'_{\text{evo}}$ only when specific success criteria are met.

The Evolutionary Loop. These operators are composed into the reflection-driven closed-loop procedure shown in Algorithm 1. Starting from an initial lifted state $\mathcal{V}_{\text{evo}}^{(0)}$, the agent first executes to collect an observational trace $\mathcal{Z}$ (tool outputs, intermediate decisions, failures, and progress signals). The reflect operator ρ maps $\mathcal{Z}$ to a set of causal hypotheses $\mathcal{H}$, which are then translated by σ into concrete modification primitives $\mathcal{D}$ (e.g., prompt edits, tool adjustments, or memory updates) over the learnable subset of $\mathcal{V}_{\text{evo}}$. The improve operator ι applies $\mathcal{D}$ via RSPL interfaces to obtain a candidate state, which is evaluated by ε to produce $\mathcal{S}$ capturing both performance metrics and safety constraints. Finally, the commit operator κ gates the transition by accepting only candidates that satisfy the predefined criteria, recording each accepted change as a versioned resource update with auditable lineage and enabling rollback when necessary.

Algorithm 1 Reflection Optimizer Evolutionary Loop

Input: Agentic System $\mathcal{A}$, Objective $\mathcal{G}$, Budget T **Output:** Optimized state $\mathcal{V}_{\text{evo}}^*$

```
1: Initialization:
2:  $\mathcal{V}_{\text{evo}}^{(0)} \leftarrow \text{VariableLifting}(\mathcal{A})$   $\triangleright$  Project resources to optimization manifold
3:  $\mathcal{Z}^{(0)} \leftarrow \text{Execute}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(0)})$   $\triangleright$  Trace: tool I/O, failures, latencies, progress
4: Optimization Cycle:
5: for  $t = 0, 1, \dots, T - 1$  do
6:   // Phase 1: Diagnosis & Proposal
7:    $\mathcal{H}^{(t)} \leftarrow \rho(\mathcal{Z}^{(t)}, \mathcal{V}_{\text{evo}}^{(t)})$   $\triangleright$  Reflect: attribute failures / inefficiencies
8:    $\mathcal{D}^{(t)} \leftarrow \sigma(\mathcal{V}_{\text{evo}}^{(t)}, \mathcal{H}^{(t)})$   $\triangleright$  Select: propose edits over learnable variables
9:   // Phase 2: Mutation & Verification
10:   $\tilde{\mathcal{V}}_{\text{evo}}^{(t+1)} \leftarrow \iota(\mathcal{V}_{\text{evo}}^{(t)}, \mathcal{D}^{(t)})$   $\triangleright$  Improve: apply proposed updates (candidate)
11:   $\mathcal{S}^{(t+1)} \leftarrow \varepsilon(\tilde{\mathcal{V}}_{\text{evo}}^{(t+1)}, \mathcal{G})$   $\triangleright$  Evaluate: metrics + safety invariants
12:  // Phase 3: Gating & Transition
13:  if  $\text{Accept}(\mathcal{S}^{(t+1)})$  then
14:    // Accept: safe & non-degrading
15:     $\mathcal{V}_{\text{evo}}^{(t+1)} \leftarrow \kappa(\tilde{\mathcal{V}}_{\text{evo}}^{(t+1)}, \mathcal{S}^{(t+1)})$   $\triangleright$  Commit: versioned update
16:  else
17:    // Reject: rollback / keep previous state
18:     $\mathcal{V}_{\text{evo}}^{(t+1)} \leftarrow \mathcal{V}_{\text{evo}}^{(t)}$ 
19:  end if
20:  // Phase 4: Next Iteration
21:   $\mathcal{Z}^{(t+1)} \leftarrow \text{Execute}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(t+1)})$   $\triangleright$  Re-run under updated resources
22:  if  $\text{Converged}(\mathcal{S}^{(t+1)})$  then
23:    break
24:  end if
25: end for
26: return  $\mathcal{V}_{\text{evo}}^{(t)}$ 
```

E.3.5 TextGrad Optimizer

Evolvable Variables. In the TextGrad instantiation, the evolvable variables are restricted to a subset of *prompt variables* marked as optimizable and lifted into TextGrad variables with explicit role descriptions. In our implementation, each optimizable prompt module is represented as a TextGrad variable whose value is the current prompt text and whose role description specifies the prompt’s function, enabling the optimizer to condition updates on its intended semantics.

Operator Algebra. TextGrad instantiates SEPL with a prompt-level operatorization in which “gradients” are natural-language critiques produced by an LLM evaluator and updates are implemented as constrained prompt rewrites. Following the standard TextGrad view, we express the method with five core operators, namely *Execute*, *Loss*, *Backward*, *Improve*, and *Commit*, where the “gradient” is a piece of text (a critique) rather than a numeric vector:

- **Execute** (χ_{tg}). $\chi_{\text{tg}} : (\mathcal{A}, \mathcal{V}_{\text{evo}}, x, f) \rightarrow \mathcal{Z}$ runs the agent under the current prompt variables and produces an execution trace/outcome.
- **Loss** (λ_{tg}). $\lambda_{\text{tg}} : \mathcal{Z} \rightarrow \mathcal{G}_{\text{tg}}$, where $\mathcal{G}_{\text{tg}}$ is a space of natural-language critiques (textual gradients). In our implementation, λ_{tg} is realized by TextLoss, which queries an evaluator LLM and returns critique feedback.
- **Backward** (β_{tg}). $\beta_{\text{tg}} : \mathcal{V}_{\text{evo}} \times \mathcal{G}_{\text{tg}} \rightarrow \mathcal{V}_{\text{evo}}$ assigns textual gradients to optimizable prompt variables by storing the critique (optionally with context) in a per-variable gradient buffer. In our current implementation, we distribute the same critique to each optimizable prompt variable for stability.
- **Improve** (ι_{tg}). $\iota_{\text{tg}} : \mathcal{V}_{\text{evo}} \rightarrow \mathcal{V}'_{\text{evo}}$ rewrites prompt variables via a textual-gradient-descent step: it constructs an update instruction from each variable’s role description, current value, and accumulated textual gradients, then queries an optimizer LLM and extracts the improved variable text from a constrained output format.

Algorithm 2 TextGrad Prompt Optimization Loop

Input: Agentic System $\mathcal{A}$, task x , attachments f (optional), Budget K , evaluator/optimizer LLMs $M_{\text{eval}}, M_{\text{opt}}$

Output: Updated state $\mathcal{V}_{\text{evo}}^*$ (prompt variables updated via TextGrad)

```
1: // Phase 0: Setup
2: Set backward engine to  $M_{\text{eval}}$  ▷ Evaluator used by TextLoss
3:  $\mathcal{V}_{\text{evo}}^{(0)} \leftarrow \text{VariableLifting}(\mathcal{A})$  ▷ Lift optimizable prompts to TextGrad variables
4: Initialize textual optimizer with  $M_{\text{opt}}$  ▷ TextualGradientDescent over prompt vars
5: // Optimization Cycle
6: for  $k = 0, 1, \dots, K - 1$  do
7:   // Phase 1: Execute (Forward)
8:    $\mathcal{Z}^{(k)} \leftarrow \chi_{\text{tg}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(k)}, x, f)$  ▷ Run agent with current prompts
9:   // Phase 2: Loss (Textual Gradient)
10:  Build evaluation instruction from  $\mathcal{Z}^{(k)}$  ▷ Condition on success/error
11:   $g^{(k)} \leftarrow \lambda_{\text{tg}}(\mathcal{Z}^{(k)})$  ▷ TextLoss produces critique string
12:  // Phase 3: Backward (Assign Gradients)
13:   $\mathcal{V}_{\text{evo}}^{(k)} \leftarrow \beta_{\text{tg}}(\mathcal{V}_{\text{evo}}^{(k)}, g^{(k)})$  ▷ Assign critique to gradient buffers
14:  // Phase 4: Improve (Textual Gradient Descent)
15:   $\hat{\mathcal{V}}_{\text{evo}}^{(k+1)} \leftarrow \iota_{\text{tg}}(\mathcal{V}_{\text{evo}}^{(k)})$  ▷ Rewrite prompts via textual GD
16:  // Phase 5: Commit & Next Iteration
17:   $\mathcal{V}_{\text{evo}}^{(k+1)} \leftarrow \kappa_{\text{tg}}(\hat{\mathcal{V}}_{\text{evo}}^{(k+1)})$  ▷ Sync back; clear caches
18:  if  $\text{Converged}(g^{(k)})$  then
19:    break
20:  end if
21: end for
22: return  $\mathcal{V}_{\text{evo}}^{(k)}$ 
```

- **Commit** (κ_{tg}). $\kappa_{\text{tg}} : \mathcal{V}'_{\text{evo}} \rightarrow \mathcal{V}_{\text{evo}}$ synchronizes the updated prompt variables back into the running agent and clears caches, completing the state transition.

The Evolutionary Loop. Algorithm 2 presents the full TextGrad optimization cycle in operator form. At each iteration, the agent is executed under the current prompt variables to obtain a trace $\mathcal{Z}$ via χ_{tg} , an LLM-based evaluator produces a natural-language critique $g \in \mathcal{G}_{\text{tg}}$ via λ_{tg} , the critique is assigned as a *textual gradient* to the optimizable prompt variables via β_{tg} , the prompt variables are improved via ι_{tg} using textual-gradient-descent, and the candidate state is committed via κ_{tg} to synchronize the updated prompts back into the running agent (and clear caches) before the next iteration.

E.3.6 Reinforce++ Optimizer

Evolvable Variables. Reinforce++ optimizes a trainable subset of RSPL resources, focusing on prompt variables and tool implementations (native scripts, MCP tools [1], and agent skills [2]), and optionally refining the produced solution text. Our implementation follows a two stage structure: (i) update trainable variables that govern behavior (e.g., prompts and tools), and (ii) update the solution itself when enabled.

Operator Algebra. Reinforce++ is characterized by a clipped objective with an explicit penalty to a reference solution, while using reflection to translate RL signals into concrete edits. We group the method into a small set of core operators:

- **Sample** (χ_{rpp}). $\chi_{\text{rpp}} : (A, \mathcal{V}_{\text{evo}}, x, f) \rightarrow \mathcal{Z}$ samples a rollout under the current resources and yields an execution trace containing the produced answer.
- **Reward** (ε_{rpp}). $\varepsilon_{\text{rpp}} : (y^{(t)}, y^{(t-1)}, y^*, y_{\text{sft}}) \rightarrow (r^{(t)}, A^{(t)}, J^{(t)}, \pi^{(t)})$ computes the RL signal tuple from the current solution $y^{(t)}$. Here $r^{(t)}$ is a task reward comparing $y^{(t)}$ with y^* , and $\pi^{(t)}$ is a policy ratio surrogate approximated via text similarity $\eta(\cdot, \cdot)$ as $\pi^{(t)} \triangleq \eta(y^{(t-1)}, y^{(t)})$ (since token-level probability ratios are unavailable in inference-only LLM settings). We define a penalty to a reference solution y_{sft} as $\text{pen}^{(t)} \triangleq \beta |\log \max(\eta(y_{\text{sft}}, y^{(t)}), \epsilon_0)|$ and set $A^{(t)} \triangleq r^{(t)} - \text{pen}^{(t)}$.

Algorithm 3 Reinforce++ Optimization Loop

Input: Agentic System $\mathcal{A}$, task x , attachments f (optional), ground truth y^* , reference solution y_{sft} , Budget T

Output: Final solution $y^{(t)}$ and updated trainable resources $\mathcal{V}_{\text{train}}$

```
1: // Initialization
2:  $\mathcal{V}_{\text{evo}}^{(0)} \leftarrow \text{VariableLifting}(\mathcal{A})$   $\triangleright$  Lift trainable resources
3:  $\mathcal{Z}^{(0)} \leftarrow \chi_{\text{rpp}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(0)}, x, f)$   $\triangleright$  Sample once
4: Extract solution  $y^{(0)}$  from  $\mathcal{Z}^{(0)}$ 
5:  $y^{(-1)} \leftarrow y^{(0)}$   $\triangleright$  Initialize previous solution
6: for  $t = 0, 1, \dots, T - 1$  do
7:   // Phase 1: Reinforce++ reward and objective
8:    $(r^{(t)}, A^{(t)}, J^{(t)}, \pi^{(t)}) \leftarrow \varepsilon_{\text{rpp}}(y^{(t)}, y^{(t-1)}, y^*, y_{\text{sft}})$   $\triangleright$  Reward, penalty, clipped objective
9:   // Phase 2: Improve trainable resources (prompt and tool)
10:   $\mathcal{V}_{\text{train}}^{(t)} \leftarrow \text{GetTrainables}(\mathcal{V}_{\text{evo}}^{(t)})$ 
11:   $\mathcal{H}_{\text{train}}^{(t)} \leftarrow \delta_{\text{rpp}}(\mathcal{Z}^{(t)}, \mathcal{V}_{\text{train}}^{(t)}, r^{(t)}, A^{(t)}, J^{(t)}, \pi^{(t)})$   $\triangleright$  Diagnose conditioned on RL signals
12:   $\tilde{\mathcal{V}}_{\text{train}}^{(t+1)} \leftarrow \iota_{\text{rpp}}(\mathcal{V}_{\text{train}}^{(t)}, \mathcal{H}_{\text{train}}^{(t)})$   $\triangleright$  Apply edits to trainables (candidate)
13:   $\mathcal{V}_{\text{train}}^{(t+1)} \leftarrow \kappa_{\text{rpp}}(\tilde{\mathcal{V}}_{\text{train}}^{(t+1)})$   $\triangleright$  Commit updates
14:  // Phase 3: Re run under updated resources
15:   $\mathcal{Z}^{(t+1)} \leftarrow \chi_{\text{rpp}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(t)} \cup \mathcal{V}_{\text{train}}^{(t+1)}, x, f)$ 
16:  Extract solution  $y^{(t+1)}$  from  $\mathcal{Z}^{(t+1)}$ 
17:  // Phase 4: Optional solution refinement
18:   $\mathcal{H}_{\text{sol}}^{(t)} \leftarrow \delta_{\text{rpp}}(\mathcal{Z}^{(t+1)}, \{y^{(t+1)}\}, r^{(t)}, A^{(t)}, J^{(t)}, \pi^{(t)})$   $\triangleright$  Diagnose solution quality
19:   $\tilde{y}^{(t+1)} \leftarrow \iota_{\text{rpp}}(y^{(t+1)}, \mathcal{H}_{\text{sol}}^{(t)})$   $\triangleright$  Edit solution text (candidate)
20:   $y^{(t+1)} \leftarrow \kappa_{\text{rpp}}(\tilde{y}^{(t+1)})$   $\triangleright$  Commit solution update
21:  // Phase 5: Early stopping
22:  if  $\text{Satisfied}(\mathcal{Z}^{(t+1)})$  then
23:    break
24:  end if
25:   $y^{(t)} \leftarrow y^{(t+1)}$   $\triangleright$  Advance current solution
26: end for
27: return  $y^{(t)}$ 
```

The clipped Reinforce++ objective is

$$J^{(t)} \triangleq \min(\pi^{(t)} A^{(t)}, \bar{\pi}^{(t)} A^{(t)}), \quad \bar{\pi}^{(t)} \triangleq \text{clip}(\pi^{(t)}, 1 - \epsilon, 1 + \epsilon).$$

- **Diagnose** (δ_{rpp}). $\delta_{\text{rpp}} : (\mathcal{Z}, \mathcal{V}_{\text{train}}, r^{(t)}, A^{(t)}, J^{(t)}, \pi^{(t)}) \rightarrow \mathcal{H}$ produces an edit oriented diagnosis that is explicitly conditioned on the RL metrics and the execution trace.
- **Improve** (ι_{rpp}). $\iota_{\text{rpp}} : (\mathcal{V}, \mathcal{H}) \rightarrow \mathcal{V}'_{\text{evo}}$ applies RL informed edits to either (i) the trainable resources $\mathcal{V}_{\text{train}}$ such as prompts and tools, or (ii) the solution variable itself when solution refinement is enabled, yielding a candidate state.
- **Commit** (κ_{rpp}). $\kappa_{\text{rpp}} : \mathcal{V}'_{\text{evo}} \rightarrow \mathcal{V}_{\text{evo}}$ applies accepted updates back to RSPL resources, completing the state transition.

The Evolutionary Loop. Algorithm 3 summarizes the Reinforce++ loop in a phased form. Each iteration (i) computes Reinforce++ signals via the clipped objective and the penalty to the reference solution, (ii) improves trainable resources through RL conditioned reflection and edits, (iii) optionally improves the solution text, and (iv) applies an early stopping evaluation.

E.3.7 GRPO Optimizer

Evolvable Variables. GRPO optimizes a trainable subset of RSPL resources, focusing on prompt variables and tool implementations (native scripts, MCP tools [1], and agent skills [2]), and optionally refining the produced solution text. Similar to Reinforce++, our implementation follows a two stage

structure: (i) update trainable variables that govern behavior (e.g., prompts and tools), and (ii) update the solution itself when enabled.

Operator Algebra. GRPO is characterized by sampling multiple candidate solutions per step and using group normalized advantages with a clipped objective. We formalize the method with the following core operators:

- **Sample** (χ_{grpo}). $\chi_{\text{grpo}} : (A, \mathcal{V}_{\text{evo}}, x, f, K) \rightarrow \{\mathcal{Z}_i\}_{i=1}^K$ samples K independent rollouts under the current resources, yielding K execution traces each containing a candidate solution y_i .
- **Reward** ($\varepsilon_{\text{grpo}}$). $\varepsilon_{\text{grpo}} : (\{y_i\}_{i=1}^K, y^*, y^{(t-1)}) \rightarrow (\{r_i\}_{i=1}^K, \{A_i\}_{i=1}^K, \{J_i\}_{i=1}^K, \{\pi_i\}_{i=1}^K)$ computes RL signals for all K candidates. For each candidate y_i , we compute a task reward r_i comparing y_i with y^* , a policy ratio surrogate $\pi_i \triangleq \eta(y^{(t-1)}, y_i)$ approximated via text similarity $\eta(\cdot, \cdot)$ (since token-level probability ratios are unavailable in inference-only LLM settings), and a group normalized advantage A_i by normalizing rewards across the candidate set: $A_i = (r_i - \bar{r})/\sigma_r$ where $\bar{r}$ and σ_r are the mean and standard deviation of $\{r_i\}_{i=1}^K$. The GRPO clipped objective for each candidate is

$$J_i \triangleq \min(\pi_i A_i, \bar{\pi}_i A_i), \quad \bar{\pi}_i \triangleq \begin{cases} \min(\pi_i, 1 + \epsilon) & \text{if } A_i \geq 0 \\ \max(\pi_i, 1 - \epsilon) & \text{if } A_i < 0 \end{cases}.$$

- **Diagnose** (δ_{grpo}). $\delta_{\text{grpo}} : (\{\mathcal{Z}_i\}_{i=1}^K, \mathcal{V}_{\text{train}}, \{r_i, A_i, J_i, \pi_i\}_{i=1}^K) \rightarrow \mathcal{H}$ produces an edit oriented diagnosis that is explicitly conditioned on the multiple candidate solutions and their RL metrics, enabling the optimizer to identify patterns across candidates.
- **Improve** (ι_{grpo}). $\iota_{\text{grpo}} : (\mathcal{V}, \mathcal{H}) \rightarrow \mathcal{V}'_{\text{evo}}$ applies RL informed edits to either (i) the trainable resources $\mathcal{V}_{\text{train}}$ such as prompts and tools, or (ii) the solution variable itself when solution refinement is enabled, yielding a candidate state.
- **Commit** (κ_{grpo}). $\kappa_{\text{grpo}} : \mathcal{V}'_{\text{evo}} \rightarrow \mathcal{V}_{\text{evo}}$ applies accepted updates back to RSPL resources, completing the state transition.

The Evolutionary Loop. Algorithm 4 summarizes the GRPO loop in a phased form. Each iteration (i) samples K candidate solutions, (ii) computes GRPO signals via group normalized advantages and clipped objectives, (iii) improves trainable resources through multi candidate conditioned reflection and edits, (iv) optionally improves the solution text, and (v) applies an early stopping evaluation.

Algorithm 4 GRPO Optimization Loop

Input: Agentic System $\mathcal{A}$, task x , attachments f (optional), ground truth y^* , Budget T , number of candidates K

Output: Final solution $y^{(t)}$ and updated trainable resources $\mathcal{V}_{\text{train}}$

```

1: // Initialization
2:  $\mathcal{V}_{\text{evo}}^{(0)} \leftarrow \text{VariableLifting}(\mathcal{A})$   $\triangleright$  Lift trainable resources
3:  $\mathcal{Z}^{(0)} \leftarrow \chi_{\text{grpo}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(0)}, x, f, 1)$   $\triangleright$  Sample initial solution
4: Extract solution  $y^{(0)}$  from  $\mathcal{Z}^{(0)}$ 
5:  $y^{(-1)} \leftarrow y^{(0)}$   $\triangleright$  Initialize previous solution
6: for  $t = 0, 1, \dots, T - 1$  do
7:   // Phase 1: Sample multiple candidates
8:    $\{\mathcal{Z}_i^{(t)}\}_{i=1}^K \leftarrow \chi_{\text{grpo}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(t)}, x, f, K)$   $\triangleright$  Sample  $K$  rollouts
9:   Extract candidate solutions  $\{y_i^{(t)}\}_{i=1}^K$  from  $\{\mathcal{Z}_i^{(t)}\}_{i=1}^K$ 
10:  // Phase 2: GRPO reward and objective
11:   $(\{r_i^{(t)}\}_{i=1}^K, \{A_i^{(t)}\}_{i=1}^K, \{J_i^{(t)}\}_{i=1}^K, \{\pi_i^{(t)}\}_{i=1}^K) \leftarrow \varepsilon_{\text{grpo}}(\{y_i^{(t)}\}_{i=1}^K, y^*, y^{(t-1)})$ 
    $\triangleright$  Group normalized advantages, clipped objectives
12:  // Phase 3: Improve trainable resources (prompt and tool)
13:   $\mathcal{V}_{\text{train}}^{(t)} \leftarrow \text{GetTrainables}(\mathcal{V}_{\text{evo}}^{(t)})$ 
14:   $\mathcal{H}_{\text{train}}^{(t)} \leftarrow \delta_{\text{grpo}}(\{\mathcal{Z}_i^{(t)}\}_{i=1}^K, \mathcal{V}_{\text{train}}^{(t)}, \{r_i^{(t)}, A_i^{(t)}, J_i^{(t)}, \pi_i^{(t)}\}_{i=1}^K)$ 
    $\triangleright$  Diagnose conditioned on multi candidate RL signals
15:   $\tilde{\mathcal{V}}_{\text{train}}^{(t+1)} \leftarrow \iota_{\text{grpo}}(\mathcal{V}_{\text{train}}^{(t)}, \mathcal{H}_{\text{train}}^{(t)})$   $\triangleright$  Apply edits to trainables (candidate)
16:   $\mathcal{V}_{\text{train}}^{(t+1)} \leftarrow \kappa_{\text{grpo}}(\tilde{\mathcal{V}}_{\text{train}}^{(t+1)})$   $\triangleright$  Commit updates
17:  // Phase 4: Re run under updated resources
18:   $\mathcal{Z}^{(t+1)} \leftarrow \chi_{\text{grpo}}(\mathcal{A}, \mathcal{V}_{\text{evo}}^{(t)} \cup \mathcal{V}_{\text{train}}^{(t+1)}, x, f, 1)$ 
19:  Extract solution  $y^{(t+1)}$  from  $\mathcal{Z}^{(t+1)}$ 
20:  // Phase 5: Optional solution refinement
21:   $\mathcal{H}_{\text{sol}}^{(t)} \leftarrow \delta_{\text{grpo}}(\{\mathcal{Z}_i^{(t)}\}_{i=1}^K, \{y^{(t+1)}\}, \{r_i^{(t)}, A_i^{(t)}, J_i^{(t)}, \pi_i^{(t)}\}_{i=1}^K)$ 
    $\triangleright$  Diagnose solution quality using multi candidate context
22:   $\tilde{y}^{(t+1)} \leftarrow \iota_{\text{grpo}}(y^{(t+1)}, \mathcal{H}_{\text{sol}}^{(t)})$   $\triangleright$  Edit solution text (candidate)
23:   $y^{(t+1)} \leftarrow \kappa_{\text{grpo}}(\tilde{y}^{(t+1)})$   $\triangleright$  Commit solution update
24:  // Phase 6: Early stopping
25:  if  $\text{Satisfied}(\mathcal{Z}^{(t+1)})$  then
26:    break
27:  end if
28:   $y^{(t)} \leftarrow y^{(t+1)}$   $\triangleright$  Advance current solution
29: end for
30: return  $y^{(t)}$ 

```
